

Descrição do Componente

FotografoDeFaces

00. Sumário

- 01. Objetivo
- 02. Responsabilidade e não responsabilidade
- 03. Modos de operação
- 04. Modos de disparo
- 05. Ciclo de vida
- 06. Máquina de estados
- 07. Comportamento de cada estado
- 08. Face Lock
- 09. Seleção da melhor face no quiosque
- 10. Timer e estabilidade
- 11. Regras de captura manual
- 12. Tratamento da fotografia
- 13. Fotografia final
- 14. Mensagens exibidas
- 15. Eventos/callbacks
- 16. Comandos que a aplicação pode enviar
- 17. Contrato da fotografia produzida
- 18. Comportamentos de erro
- 19. Comportamento de reinício
- 20. Limites entre FotografoDeFaces e aplicação hospedeira

01. Objetivo

O **FotografoDeFACES** será um componente reutilizável para aplicações web desenvolvidas em React JS, destinado à captura assistida de fotografias faciais com qualidade adequada para posterior utilização em processos biométricos ou outras finalidades definidas pela aplicação hospedeira.

O componente deverá ser concebido como uma unidade de software **independente, reutilizável e desacoplada das regras de negócio das aplicações que o utilizarem**, podendo ser incorporado a diferentes projetos React sem que sua lógica interna precise ser reproduzida ou adaptada em cada projeto.

Sua responsabilidade será controlar o processo de captura facial, incluindo a utilização da câmera, detecção e avaliação das faces, orientação do usuário, verificação das condições mínimas de qualidade, seleção da face adequada quando necessário, controle de estabilidade, captura e preparação da fotografia final.

O componente deverá produzir uma fotografia facial **tratada, recortada e preparada para utilização pela aplicação hospedeira**, disponibilizando-a para que esta decida quando, como e para qual finalidade a fotografia será utilizada, inclusive para eventual envio ao seu backend.

Existe uma face candidata identificável e estável, suficientemente enquadrada, nítida, iluminada e frontal, dentro dos limites definidos pelo componente para produzir uma fotografia facial adequada ao processamento posterior.

O **FotografoDeFACES** não realizará o reconhecimento ou a identificação da pessoa fotografada, não realizará o processamento biométrico definitivo e não será responsável pelo envio da fotografia ao backend ou pela execução de regras de negócio da aplicação hospedeira.

O componente deverá operar nos modos **autorretrato, quiosque e assistido**, permitindo captura manual ou automática conforme sua configuração. Estes modos serão definidos posteriormente ao longo deste documento.

Sua arquitetura deverá permitir que as tecnologias, bibliotecas e dependências necessárias ao funcionamento interno do componente sejam encapsuladas em sua própria distribuição, de modo que a aplicação consumidora utilize uma **interface pública bem definida**, sem necessidade de conhecer ou manipular diretamente sua implementação interna.

O **FotografoDeFACES** deverá ser projetado desde sua origem para posterior **empacotamento e distribuição como componente reutilizável**, preferencialmente por meio do ecossistema de pacotes JavaScript/NPM, permitindo sua instalação e utilização em diferentes projetos React.

A forma de empacotamento, gerenciamento de dependências, definição de dependências de produção e de desenvolvimento, construção do pacote, distribuição, versionamento e mecanismo de integração com os projetos consumidores serão definidos posteriormente na especificação técnica.

O objetivo final é que o desenvolvedor de uma aplicação consumidora possa incorporar o **FotografoDeFACES** ao seu projeto, configurá-lo por meio de sua interface pública e receber a fotografia produzida pelo componente, **sem precisar conhecer ou reproduzir sua complexidade interna**.

02. Responsabilidade e não responsabilidade

2.1. Responsabilidades

O **FotografoDeFACES** será responsável exclusivamente pelo processo de obtenção e preparação de uma fotografia facial adequada para utilização posterior pela aplicação hospedeira.

São responsabilidades do componente:

1. **Gerenciar a câmera**
 - o Solicitar e utilizar o acesso à câmera disponibilizado pelo navegador;
 - o Inicializar e encerrar a utilização da câmera conforme o ciclo de vida do componente;
 - o Trabalhar com as características de imagem necessárias para a análise e captura.
2. **Detectar faces**
 - o Identificar faces presentes no campo de visão da câmera;
 - o Determinar as características necessárias para avaliar a qualidade da captura;
 - o Considerar as particularidades de cada modo de operação.
3. **Avaliar a qualidade da captura**
 - o Verificar a existência e a quantidade de faces relevantes;
 - o Avaliar enquadramento;
 - o Avaliar distância aproximada;
 - o Avaliar pose;
 - o Avaliar nitidez;
 - o Avaliar iluminação;
 - o Avaliar estabilidade;
 - o Verificar se será possível produzir o recorte final adequado.
4. **Orientar o usuário**

O **FotografoDeFaces** será responsável por orientar visualmente a pessoa durante o processo de captura, apresentando informações claras e objetivas sobre a situação atual e, quando necessário, indicando as ações que devem ser realizadas para que a fotografia alcance as condições necessárias.

A orientação poderá incluir, entre outras:

- o indicar que nenhuma face foi identificada;
- o orientar o usuário a aproximar-se ou afastar-se da câmera;
- o orientar o usuário a mover o rosto para a esquerda, direita, para cima ou para baixo;
- o orientar o usuário a posicionar o rosto adequadamente;
- o solicitar que olhe diretamente para a câmera;
- o informar quando a iluminação estiver inadequada;
- o solicitar que permaneça imóvel quando houver instabilidade;
- o informar quando a fotografia estiver apta para captura;
- o apresentar a contagem regressiva quando houver captura automática configurada;
- o informar que a fotografia está sendo capturada ou preparada;
- o informar o resultado da captura e a disponibilidade da fotografia final.

O componente poderá disponibilizar um **guia visual de enquadramento**, representado preferencialmente por uma área oval sobre a imagem da câmera, para auxiliar o usuário a posicionar adequadamente o rosto. Esse guia deverá ser controlado por uma propriedade específica do componente, permitindo que a aplicação escolha se deseja exibi-lo ou ocultá-lo.

A propriedade será denominada **showFramingGuide** e deverá aceitar valor booleano:

- o `true`: exibe o guia de enquadramento;
- o `false`: não exibe o guia de enquadramento.

O guia de enquadramento terá finalidade exclusivamente orientativa e não substituirá os critérios técnicos utilizados pelo componente para determinar se uma fotografia está apta para captura.

Embora seja especialmente útil no modo **autorretrato**, o guia poderá também ser utilizado no modo **assistido**, caso a aplicação considere conveniente. No modo **quiosque**, sua utilização poderá ser desnecessária, uma vez que o componente deverá identificar e selecionar automaticamente a melhor face candidata.

As mensagens e informações apresentadas ao usuário pelo componente também deverão ser disponibilizadas à aplicação hospedeira por meio de sua interface pública, permitindo que a aplicação reproduza ou apresente essas informações em outras regiões da tela, como cabeçalhos, rodapés, áreas de instrução ou outros elementos de sua própria interface.

Durante o processamento de uma fotografia já capturada, o componente deverá impedir uma nova ação de disparo, mantendo o controle de captura bloqueado até que o processamento seja concluído ou cancelado. A aplicação hospedeira poderá solicitar o reinício da sessão por meio da interface de controle do componente, momento em que o processo será reiniciado e o controle de captura poderá novamente ser disponibilizado quando as condições necessárias forem atendidas.

5. Selecionar a face candidata

- o Nos modos em que somente uma face for esperada, aplicar os critérios correspondentes;
- o No modo quiosque, quando houver múltiplas faces, avaliar as candidatas e selecionar uma única face para a sessão de captura;
- o Manter a candidata selecionada durante a sessão por meio do mecanismo de **Face Lock**.

6. Controlar o momento da captura

- o Impedir a captura enquanto os requisitos mínimos não forem atendidos;
- o Permitir captura manual somente quando a fotografia estiver apta;
- o Realizar captura automática quando configurada e quando todas as condições necessárias forem satisfeitas;
- o Controlar eventual contagem regressiva e interrompê-la quando a estabilidade ou outra condição necessária for perdida.

7. Preparar a fotografia

- o Selecionar a imagem apropriada para a captura;
- o Realizar o recorte facial definido pelo componente;
- o Redimensionar a imagem quando necessário;
- o Aplicar tratamento e compressão adequados;
- o Produzir a fotografia final no formato definido pelo componente.

8. Apresentar e disponibilizar a fotografia

- o Exibir a fotografia final produzida pelo próprio componente;
- o Manter a fotografia apresentada até que a aplicação determine o reinício ou refazimento da captura;
- o Disponibilizar a fotografia final à aplicação hospedeira por meio da interface pública do componente.

9. Comunicar seu estado à aplicação

- o Informar alterações de estado;
- o Informar alterações na qualidade da captura;
- o Informar mensagens e orientações relevantes;
- o Informar a conclusão de uma fotografia;
- o Informar erros e situações excepcionais;
- o Disponibilizar, por meio da interface pública e dos mecanismos de controle definidos, informações sobre o estado atual do componente.

2.2. Não responsabilidades

O **FotografoDeFaces** não será responsável por atividades que pertençam à aplicação hospedeira ou ao seu backend.

Não são responsabilidades do componente:

10. **Reconhecer ou identificar a pessoa**
 - o Não deverá determinar quem é a pessoa fotografada;
 - o Não deverá comparar a fotografia com uma identidade cadastrada;
 - o Não deverá realizar reconhecimento facial ou autenticação biométrica definitiva.
11. **Executar o processamento biométrico definitivo**
 - o O componente poderá avaliar se a fotografia apresenta condições adequadas para processamento posterior, mas não deverá executar o cálculo biométrico que será realizado pelo backend ou por outro serviço.
12. **Enviar a fotografia ao backend**
 - o O componente deverá disponibilizar a fotografia final à aplicação;
 - o Caberá à aplicação hospedeira decidir quando, para onde e como enviar essa fotografia.
13. **Executar regras de negócio**
 - o Não deverá decidir se uma pessoa pode registrar ponto;
 - o Não deverá determinar se um registro de ponto é válido;
 - o Não deverá controlar jornada, expediente, matrícula, usuário, permissões ou qualquer outra regra específica da aplicação hospedeira.
14. **Controlar o fluxo posterior à captura**
 - o Após produzir e disponibilizar a fotografia, o componente deverá permanecer no estado correspondente até receber da aplicação a instrução para iniciar, refazer ou encerrar uma nova sessão, conforme definido posteriormente neste documento.
15. **Conhecer a estrutura interna da aplicação hospedeira**
 - o Não deverá depender de componentes, serviços, rotas, estados globais, APIs ou regras específicas de qualquer aplicação consumidora.
16. **Exigir que a aplicação conheça sua implementação interna**
 - o A aplicação hospedeira deverá interagir com o FotografoDeFaces exclusivamente por sua interface pública;
 - o Bibliotecas, modelos, algoritmos e demais mecanismos utilizados internamente deverão permanecer encapsulados conforme definido posteriormente na especificação técnica.
17. **Definir a política de armazenamento da fotografia**
 - o O componente produzirá e disponibilizará a fotografia final, mas a decisão sobre armazenamento permanente, descarte, persistência, transmissão ou utilização posterior será responsabilidade da aplicação hospedeira.

03. Modos de operação

O **FotografoDeFACES** deverá disponibilizar três modos de operação: **autorretrato**, **quiosque** e **assistido**.

O modo de operação determinará principalmente a forma como o componente deverá interpretar as faces presentes diante da câmera, os critérios de seleção da face candidata, o nível de tolerância de posicionamento e a forma de interação esperada com a pessoa fotografada.

O modo deverá ser definido pela aplicação hospedeira por meio de propriedade do componente e poderá ser alterado durante a execução da aplicação. Quando uma alteração de modo ocorrer durante uma sessão de captura, o componente deverá adequar seu comportamento ao novo modo, preservando a consistência de seu ciclo de vida e, quando necessário, reiniciando a sessão de captura.

3.1. Modo autorretrato

O modo **autorretrato** será destinado a situações nas quais a própria pessoa fotografada opera o dispositivo e posiciona seu rosto diante da câmera.

Nesse modo:

- espera-se normalmente a presença de **uma única face**;
- a presença de mais de uma face deverá impedir a captura;
- o componente deverá orientar a pessoa para que se posicione adequadamente;
- os critérios de enquadramento, posicionamento, distância e pose serão aplicados de maneira mais rigorosa;
- o **guia de enquadramento**, quando habilitado por **showFramingGuide**, deverá auxiliar visualmente a pessoa a posicionar o rosto;
- a pessoa poderá realizar uma captura manual ou aguardar a captura automática, conforme a configuração do componente;
- a fotografia somente poderá ser capturada quando todos os requisitos mínimos de qualidade forem atendidos.

O modo autorretrato será adequado, por exemplo, para uma pessoa utilizando seu próprio computador ou dispositivo para realizar seu registro.

3.2. Modo quiosque

O modo **quiosque** será destinado a equipamentos instalados em locais de atendimento ou autoatendimento, como computadores posicionados em áreas comuns dos prédios da SEAGRI, equipados com câmera e utilizados por diferentes pessoas ao longo do tempo.

Nesse modo:

- poderá haver **zero, uma ou várias faces** simultaneamente no campo de visão da câmera;
- a existência de múltiplas faces não deverá, por si só, impedir a captura;
- o componente deverá avaliar as faces disponíveis e selecionar uma única face candidata que apresente as melhores condições para captura;
- após a seleção da face candidata, deverá ser aplicado o mecanismo de **Face Lock**, impedindo a substituição do candidato durante a sessão de captura;
- somente a face selecionada deverá ser utilizada na fotografia final;
- as demais faces presentes no campo de visão não deverão fazer parte da fotografia produzida;
- os critérios de enquadramento, posicionamento, distância e pose deverão possuir uma **tolerância adicional automática**, considerando que o usuário de um quiosque terá menor precisão para se posicionar diante da câmera;
- essa tolerância será determinada internamente pelo componente e não deverá exigir que a aplicação hospedeira forneça parâmetros específicos para cada captura;
- os critérios relacionados à qualidade efetiva da fotografia, como nitidez, iluminação e viabilidade do recorte final, não deverão ser flexibilizados de forma que comprometam a qualidade mínima necessária;
- o componente deverá orientar a pessoa selecionada durante o processo de captura;
- o componente deverá permanecer preparado para uma nova pessoa somente quando a aplicação hospedeira determinar que a sessão anterior foi encerrada e solicitar o reinício do processo.

A seleção de uma face no modo quiosque deverá considerar a **qualidade da candidata**, e não simplesmente sua proximidade em relação à câmera.

O componente deverá ser capaz de identificar a melhor candidata dentre as faces presentes e, após selecioná-la, manter essa escolha durante a sessão por meio do **Face Lock**.

O modo quiosque deverá ser especialmente adequado para utilização em um computador permanentemente instalado, podendo ser utilizado em conjunto com uma apresentação em **tela cheia**, permitindo que o equipamento funcione como um ponto de registro dedicado.

3.3. Modo assistido

O modo **assistido** será destinado a situações nas quais outra pessoa, como um operador, estará auxiliando o indivíduo fotografado.

Nesse modo:

- o operador poderá orientar diretamente a pessoa diante da câmera;
- os critérios de qualidade continuarão sendo aplicados pelo componente;

- o componente deverá informar visualmente quando a fotografia estiver apta;
- o operador poderá decidir o momento do disparo quando a captura manual estiver configurada;
- o componente não deverá permitir a captura manual enquanto os requisitos mínimos de qualidade não forem atendidos;
- o **guia de enquadramento**, quando habilitado por **showFramingGuide**, poderá ser utilizado como referência visual para auxiliar o posicionamento;
- a tolerância de posicionamento não deverá ser automaticamente ampliada como ocorre no modo quiosque, salvo definição posterior na especificação técnica.

O modo assistido será adequado, por exemplo, para atendimento presencial, cadastro acompanhado por funcionário ou situações em que um operador esteja responsável por auxiliar o usuário.

3.4. Comparação dos modos

Característica	Autorretrato	Quiosque	Assistido
Uso principal	Pessoa fotografa a si mesma	Autoatendimento coletivo	Operador auxilia a pessoa
Faces simultâneas	Normalmente 1	0, 1 ou várias	Definido pela situação
Seleção automática de candidata	Não aplicável quando houver mais de uma	Sim	Conforme necessidade
Face Lock	Quando aplicável	Obrigatório após seleção	Quando aplicável
Tolerância de posicionamento	Padrão	Ampliada automaticamente	Padrão
Guia de enquadramento	Pode ser exibido	Normalmente desnecessário	Pode ser exibido
Captura manual	Sim	Opcional conforme configuração	Sim
Captura automática	Sim	Sim	Sim
Tela cheia	Opcional	Especialmente relevante	Opcional
Reinício controlado pela aplicação	Sim	Obrigatório para nova pessoa	Sim

A diferenciação entre os modos deverá permanecer encapsulada no **FotografoDeFaces**. A aplicação hospedeira deverá apenas informar o modo desejado e utilizar a interface pública do componente, sem precisar reproduzir suas regras internas de seleção, avaliação ou captura.

04. Modos de disparo

O FotografoDeFaces deverá permitir dois modos fundamentais de disparo: manual e automático, sendo que o modo automático poderá ocorrer imediatamente ou após um período configurável.

O comportamento será determinado pela propriedade `autoCaptureAfter`.

Essa propriedade poderá assumir três situações:

4.1 `autoCaptureAfter = null`

Indica disparo manual.

O componente deverá:

- realizar normalmente a detecção e avaliação da face;
- conduzir o usuário até o estado PRONTO;
- informar visualmente que a fotografia está apta para captura;
- disponibilizar o botão de captura no próprio componente;
- permitir também que a aplicação solicite o disparo por meio de `capture()` pelo `useRef`.

O disparo somente poderá ocorrer quando o componente estiver em PRONTO.

No modo manual, portanto, o próprio componente poderá oferecer ao usuário o botão de disparo, enquanto a aplicação também poderá, opcionalmente, comandar a captura por meio do `useRef`.

4.2 `autoCaptureAfter = 0`

Indica disparo automático imediato.

Assim que o componente atingir PRONTO, deverá iniciar automaticamente a captura, sem aguardar um período adicional.

O fluxo será:

AGUARDANDO

↓

DETECTANDO

↓

AVALIANDO

↓

PRONTO

↓

CAPTURANDO

4.3 autoCaptureAfter > 0

Indica disparo automático temporizado.

Quando a fotografia atingir as condições necessárias para o estado PRONTO, o componente deverá iniciar o estado CRONOMETRANDO.

Durante esse período, deverá apresentar visualmente ao usuário a contagem regressiva, permitindo que ele compreenda que a captura está prestes a ocorrer.

O fluxo será:

AGUARDANDO



DETECTANDO



AVALIANDO



PRONTO



CRONOMETRANDO



CAPTURANDO

Por exemplo, para:

```
autoCaptureAfter={3000}
```

o componente poderá apresentar mensagens como:

Aguarde 3 segundos.

Aguarde 2 segundos.

Aguarde 1 segundo.

Ao término do período, a fotografia será capturada.

4.4 Interrupção durante o disparo automático

A condição de PRONTO deverá permanecer válida durante o processo de disparo automático.

Caso, durante CRONOMETRANDO, a face deixe de atender aos critérios necessários — por exemplo, devido a movimento significativo, perda de enquadramento, alteração excessiva da pose ou perda da face candidata — o cronômetro deverá ser interrompido.

O componente não deverá produzir uma fotografia inadequada.

A perda efetiva da candidata deverá fazer o componente retornar a AGUARDANDO, iniciando posteriormente um novo ciclo de busca e avaliação.

No modo quiosque, essa regra é especialmente importante: uma vez selecionada uma face candidata, ela deverá permanecer protegida pelo Face Lock durante o processo.

Outra face presente no ambiente não poderá assumir o lugar da candidata enquanto o ciclo estiver em andamento.

4.5 O disparo nunca ignora o estado de qualidade

Independentemente da forma de disparo, o `FotografoDeFACES` não deverá produzir uma fotografia enquanto não tiver condições de considerar a captura adequada.

Assim, o comando:

```
fotografoRef.current.capture()
```

não significa "capture imediatamente a qualquer custo".

Significa solicitar uma captura, que somente será efetivada se o componente estiver em PRONTO.

No modo automático, o próprio componente realizará essa solicitação internamente ao atingir as condições determinadas por `autoCaptureAfter`.

No modo manual, o usuário poderá utilizar o botão de captura apresentado pelo próprio componente ou, alternativamente, a aplicação poderá utilizar `capture()` por meio do `useRef`.

Se `capture()` for solicitado quando o componente não estiver em PRONTO, não deverá ser produzida uma fotografia inadequada. O comportamento específico dessa solicitação será definido na especificação da API pública.

4.6 Regra geral de disparo

Independentemente do modo escolhido, a regra fundamental será:

O disparo somente poderá ocorrer quando o `FotografoDeFACES` considerar a fotografia PRONTO.

A diferença entre os modos estará apenas em quem inicia o disparo e em quanto tempo decorre entre o estado PRONTO e a captura:

<code>autoCaptureAfter</code>	Comportamento
<code>null</code>	Manual
<code>0</code>	Automático imediato
<code>> 0</code>	Automático com temporização

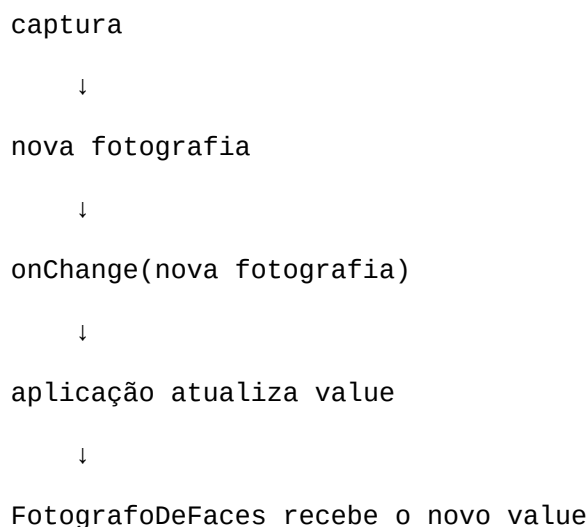
Essa estrutura mantém o comportamento simples, previsível e reutilizável nos três modos de operação do componente: autorretrato, quiosque e assistido.

4.7 Revisão da fotografia após a captura

Após uma captura bem-sucedida, o componente deverá entrar em `FOTOGRAFIA_PRONTA`.

O valor público `value` deverá representar sempre o valor atual do componente, independentemente do estado em que ele se encontre.

Assim, uma nova fotografia produzida poderá ser imediatamente atribuída ao `value` por meio do fluxo controlado do componente:



Quando a funcionalidade de revisão estiver habilitada pela propriedade `reviewFor`, o componente deverá permitir que o usuário decida o que fazer com a fotografia atualmente exibida.

A funcionalidade deverá disponibilizar, conforme o estado do componente:

- Trocar;
- Limpar;
- Confirmar;

- Cancelar.

As ações Confirmar e Cancelar somente deverão ser apresentadas após o usuário ou a aplicação ter iniciado uma operação de alteração por meio de Trocar ou Limpar.

O componente deverá manter internamente uma referência privada denominada `value_rollback`, destinada exclusivamente à possibilidade de restaurar o valor anterior durante uma operação de alteração.

`value_rollback` não representa o valor público do componente e não deverá ser disponibilizado como propriedade pública.

4.8 Comportamento de `reviewFor`

A propriedade poderá assumir três situações:

`reviewFor={null}`

Não haverá janela de revisão após a captura. As ações Trocar, Limpar, Confirmar e Cancelar não serão disponibilizadas pelo componente.

`reviewFor={0}`

A janela de revisão permanecerá disponível indefinidamente, até que o usuário ou a aplicação tome uma decisão.

`reviewFor={3000}`

A janela de revisão permanecerá disponível durante 3 segundos.

O período definido por `reviewFor` começará após a fotografia ter sido capturada e disponibilizada, e não durante sua preparação.

O mesmo período será utilizado para as ações de revisão disponibilizadas após a captura.

Quando o período de revisão terminar, as ações correspondentes deverão desaparecer e não haverá mais ação a ser executada naquela oportunidade.

Uma nova operação somente poderá ocorrer mediante novo ciclo do componente.

4.9 Troca da fotografia

Quando `reviewFor` estiver habilitado e houver uma fotografia atualmente exibida em `FOTOGRAFIA_PRONTA`, a ação Trocar deverá permitir que o usuário descarte o valor atual e inicie um novo ciclo de captura.

Antes de iniciar a troca, o componente deverá preservar internamente o valor atual:

```
value_rollback = value atual
```

Em seguida, deverá solicitar à aplicação a alteração do valor para:

```
onChange(null)
```

A aplicação atualizará o value, fazendo com que o componente passe a operar com value = null e retorne ao fluxo de captura.

Quando uma nova fotografia for produzida, ela passará a ser o novo value por meio de onChange.

Durante esse processo, o value_rollback continuará preservando a fotografia anterior.

O fluxo será:

Fotografia A

```
value = A
```

```
value_rollback = null
```

↓ Trocar

```
value = null
```

```
value_rollback = A
```

↓ nova captura

```
value = B
```

```
value_rollback = A
```

↓

[Confirmar] [Cancelar]

Nesse momento, `value` e `value_rollback` poderão simultaneamente conter fotografias diferentes. Essa situação é válida e representa uma operação de alteração em andamento.

4.10 Limpeza da fotografia

Quando `reviewFor` estiver habilitado, a ação `Limpar` deverá permitir que o usuário remova o valor atualmente exibido.

Antes da limpeza, o componente deverá preservar internamente o valor atual:

```
value_rollback = value atual
```

Em seguida, deverá solicitar:

```
onChange(null)
```

O componente passará novamente a operar com `value = null` e seguirá o fluxo normal de captura.

Quando uma nova fotografia for produzida, ela será atribuída ao `value` por meio de `onChange`.

Assim como na ação `Trocar`, o `value_rollback` permanecerá preservado durante a operação.

Portanto, tanto `Trocar` quanto `Limpar` poderão iniciar uma operação que posteriormente apresentará:

```
[ Confirmar ] [ Cancelar ]
```

A diferença entre as duas ações está apenas na intenção inicial:

- `Trocar` expressa a intenção de substituir a fotografia;
- `Limpar` expressa a intenção de remover a fotografia e, se desejado, produzir posteriormente uma nova.

4.11 Confirmação da fotografia

A ação `Confirmar` deverá encerrar a operação de alteração e estabelecer o valor atualmente exibido como o novo valor válido do componente.

Como `value` já representa o valor atual do componente, a confirmação não deverá exigir uma segunda fotografia ou outro valor paralelo.

Ao confirmar:


```
value = fotografia atualmente exibida  
value_rollback = null
```

A partir desse momento, uma futura operação de Trocar ou Limpar deverá considerar esse `value` como o valor anterior a ser preservado em `value_rollback`.

A regra fundamental será:

O compromisso do componente é sempre com o valor atualmente representado por `value`, enquanto `value_rollback` existe somente durante uma operação de alteração e permite seu eventual cancelamento.

4.12 Cancelamento da alteração

A ação Cancelar deverá abandonar a alteração em andamento e restaurar o valor existente em `value_rollback`.

O componente deverá solicitar à aplicação:

```
onChange(value_rollback)
```

Após a aplicação atualizar o `value`, o componente deverá limpar sua referência interna:

```
value_rollback = null
```

O fluxo será:

```
value = B
```

```
value_rollback = A
```

↓ Cancelar

```
onChange(A)
```

↓

```
value = A
```

```
value_rollback = null
```

Quando a operação tiver iniciado sem uma fotografia anterior, `value_rollback` poderá conter `null`. Nesse caso, Cancelar deverá restaurar o componente ao estado sem fotografia:

```
value = null
```

```
value_rollback = null
```

4.13 Relação entre as ações de revisão

Quando uma fotografia estiver disponível em `FOTOGRAFIA_PRONTA` e `reviewFor` estiver habilitado, o componente deverá disponibilizar inicialmente:

```
[ Trocar ] [ Limpar ]
```

A execução de qualquer uma dessas ações iniciará uma operação de alteração e fará com que o componente preserve o valor anterior em `value_rollback`.

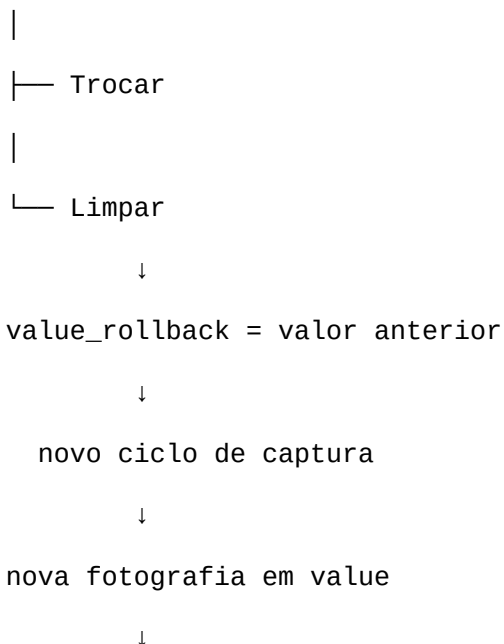
Após uma nova fotografia ser produzida, o componente deverá disponibilizar:

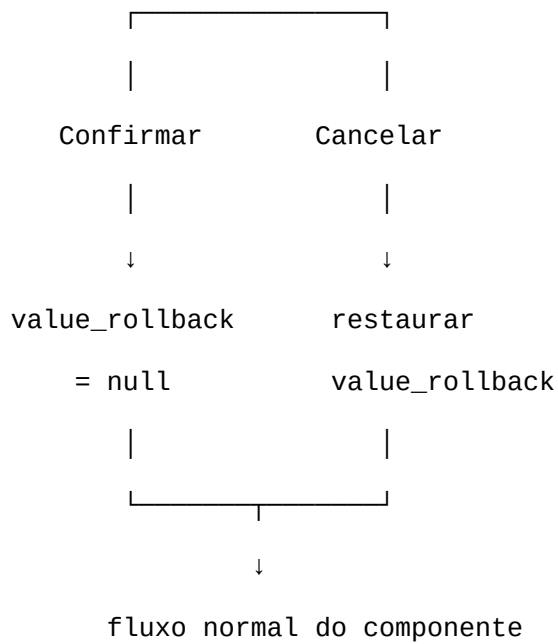
```
[ Confirmar ] [ Cancelar ]
```

Essa regra também será válida quando o valor anterior for `null`.

O fluxo geral será:

`FOTOGRAFIA_PRONTA`





O `reviewFor` deverá controlar a duração dessa oportunidade de interação. Encerrada a janela de revisão, as ações correspondentes deverão desaparecer e aquela oportunidade será encerrada.

A aplicação poderá iniciar uma nova operação posteriormente, conforme o contexto em que o componente estiver sendo utilizado.

4.14 Regra de valor do componente

O `value` deverá ser tratado como o valor atual e real do `FotografoDeFACES`, seguindo o padrão de um componente controlado do React.

A aplicação deverá fornecer o valor ao componente por meio de:

```
value={foto}
```

e receber as alterações por meio de:

```
onChange={setFoto}
```

O componente não deverá considerar que `value` representa exclusivamente uma fotografia "confirmada". Ele poderá representar:

- `null`, quando não houver fotografia atualmente disponível;
- uma fotografia carregada inicialmente pela aplicação;
- uma fotografia recém-produzida;
- uma fotografia que esteja sendo utilizada durante uma operação de alteração.

A máquina de estados deverá reagir ao valor atual recebido por `value`, e não substituir unilateralmente esse valor.

A relação entre `value`, `onChange`, `value_rollback` e os estados do componente deverá ser detalhada nos capítulos correspondentes.

05. Ciclo de vida

O ciclo de vida do `FotografoDeFaces` deverá representar, de forma previsível, todo o processo desde o momento em que o componente está disponível para receber uma pessoa até a produção, disponibilização, alteração ou restauração de uma fotografia.

O ciclo deverá ser independente do sistema hospedeiro. A aplicação deverá controlar o valor do componente por meio da propriedade `value` e receber suas alterações por meio de `onChange`.

O estado do componente deverá ser determinado em função das condições atuais de operação e do valor recebido em `value`. O componente não deverá substituir unilateralmente o `value` fornecido pela aplicação.

5.1 Início do componente

Ao ser apresentado, o componente deverá analisar seu estado inicial e o valor recebido em `value`.

Quando `value = null`

O componente deverá iniciar sem uma fotografia atualmente disponível e permanecer apto a iniciar um novo processo de captura.

O estado inicial será:

AGUARDANDO

A partir desse estado, o componente poderá iniciar a detecção de faces conforme o modo de operação configurado.

Quando `value` contém uma fotografia

O componente deverá considerar essa fotografia como o valor atual do componente.

Nesse caso, deverá apresentar a fotografia de acordo com o comportamento visual definido para `FOTOGRAFIA_PRONTA`, sem iniciar automaticamente uma nova captura.

A aplicação poderá permitir que essa fotografia seja alterada conforme a configuração de `reviewFor`.

5.2 Busca por uma candidata

Quando `value = null` e o componente estiver disponível para captura, o ciclo deverá iniciar a busca por uma face candidata.

Nos modos autorretrato e assistido, o componente deverá considerar válida para o processo uma situação em que exista exatamente uma face detectada no campo de visão. A presença de nenhuma face ou de mais de uma face impedirá que o processo avance para PRONTO.

O modo quiosque possui uma dinâmica diferente. Nesse modo, o componente deverá admitir a presença simultânea de várias pessoas no campo de visão da câmera. A existência de múltiplas faces, portanto, não será por si só uma condição de erro.

O componente deverá selecionar uma única face candidata com base nos critérios de qualidade definidos para o modo quiosque. Após a seleção, deverá aplicar o mecanismo de Face Lock, mantendo essa pessoa como único alvo do ciclo de captura.

A partir desse momento, as demais faces presentes no ambiente deverão ser desconsideradas para fins de avanço do ciclo. Somente a face candidata selecionada poderá evoluir para PRONTO.

Enquanto o Face Lock estiver ativo, o componente deverá monitorar continuamente a candidata selecionada. Caso ela deixe de atender às condições necessárias para continuidade do processo, o componente deverá seguir as regras estabelecidas para perda da candidata, sem transferir automaticamente o foco para outra pessoa.

Somente quando o ciclo retornar a AGUARDANDO poderá ocorrer uma nova seleção de candidato.

Assim, a regra para avanço do ciclo será:

- autorretrato: exatamente uma face detectada e válida;
- assistido: exatamente uma face detectada e válida;
- quiosque: uma única face candidata selecionada e protegida pelo Face Lock, ainda que existam outras faces no campo de visão.

5.3 Preparação para captura

Após identificar uma candidata válida, o componente deverá avaliar continuamente sua qualidade.

Quando todos os critérios necessários forem atendidos, a candidata será considerada apta e o componente entrará em:

PRONTO

Nesse momento, a fotografia poderá ser capturada de acordo com `autoCaptureAfter`.

A condição `PRONTO` deverá representar uma decisão do próprio componente de que a imagem atual possui qualidade suficiente para produzir a fotografia final.

5.4 Disparo

A partir de `PRONTO`, o ciclo poderá seguir diferentes caminhos.

Captura manual

`PRONTO`

↓

`CAPTURANDO`

O disparo poderá ocorrer pelo botão apresentado pelo próprio componente ou por solicitação da aplicação através do `useRef`.

Captura automática imediata

Quando:

`autoCaptureAfter={0}`

o componente deverá passar automaticamente de:

`PRONTO` → `CAPTURANDO`

Captura automática temporizada

Quando:

`autoCaptureAfter={3000}`

por exemplo:

`PRONTO`

↓

`CRONOMETRANDO`

↓

`CAPTURANDO`

Durante CRONOMETRANDO, a condição de qualidade deverá continuar sendo monitorada.

5.5 Produção da fotografia

No estado CAPTURANDO, o componente deverá produzir a fotografia final a partir da imagem da câmera.

A fotografia deverá ser:

- selecionada a partir da face candidata;
- recortada;
- enquadrada;
- redimensionada;
- orientada adequadamente;
- submetida aos tratamentos necessários;
- convertida para o formato definido pelo contrato do componente.

Concluído o processamento, o componente deverá apresentar a fotografia produzida ao usuário e disponibilizá-la como o novo valor do componente por meio de onChange.

O fluxo será:

CAPTURANDO

↓

nova fotografia produzida

↓

onChange(nova fotografia)

↓

aplicação atualiza value

↓

value = nova fotografia

↓

FOTOGRAFIA_PRONTA

5.6 Alteração do valor atual

Quando reviewFor estiver habilitado, a fotografia atualmente representada por value poderá ser alterada pelo usuário.

Antes de iniciar uma operação de Trocar ou Limpar, o componente deverá preservar internamente o valor atual:

```
value_rollback = value
```

Em seguida, deverá solicitar à aplicação a alteração do valor para `null`:

```
onChange(null)
```

O componente passará a operar com `value = null` e iniciará um novo ciclo de captura.

Quando uma nova fotografia for produzida, o componente deverá solicitar sua atribuição por meio de:

```
onChange(nova_fotografia)
```

A aplicação atualizará `value`, que passará a representar a nova fotografia atualmente exibida.

Durante essa operação, `value_rollback` continuará preservando o valor anterior.

Assim, durante uma alteração, poderá ocorrer legitimamente:

```
value = nova_fotografia
```

```
value_rollback = fotografia_anterior
```

Essa situação representa uma alteração em andamento.

5.7 Confirmação

Quando uma nova fotografia estiver sendo exibida após uma operação de alteração, o componente poderá apresentar a opção Confirmar.

Ao confirmar, o valor atual representado por `value` permanecerá como o valor do componente e a referência privada de rollback deverá ser eliminada:

```
value = fotografia_atual
```

```
value_rollback = null
```

A partir desse momento, uma futura operação de Trocar ou Limpar deverá preservar esse `value` em `value_rollback`.

5.8 Cancelamento

Quando houver uma operação de alteração em andamento, o componente poderá apresentar a opção Cancelar.

Ao cancelar, o componente deverá solicitar à aplicação a restauração do valor anteriormente preservado:

```
onChange(value_rollback)
```

Após a aplicação atualizar `value`, o componente deverá limpar sua referência interna:

```
value_rollback = null
```

Assim:

```
value = nova fotografia
```

```
value_rollback = fotografia anterior
```

↓ Cancelar

```
onChange(fotografia anterior)
```

↓

```
value = fotografia anterior
```

```
value_rollback = null
```

Quando `value_rollback = null`, o cancelamento deverá restaurar o componente para a condição sem fotografia:

```
value = null
```

```
value_rollback = null
```

5.9 Retorno ao estado de espera

O componente deverá retornar a AGUARDANDO sempre que o `value` atual for `null` e não houver uma operação de revisão que esteja conduzindo o componente para outro estado.

A partir de AGUARDANDO, uma nova busca poderá começar.

Essa regra é especialmente importante no modo quiosque. Quando houver perda efetiva da candidata protegida pelo `Face Lock`, o componente deverá retornar a:

AGUARDANDO

e somente então iniciar um novo ciclo:

AGUARDANDO

↓

DETECTANDO

Nesse novo ciclo, outra pessoa poderá se tornar a candidata.

5.10 Regra fundamental do ciclo

O `FotografoDeFACES` deverá manter uma separação clara entre:

- o valor atual do componente, representado por `value`;
- a fotografia anteriormente existente, quando necessária para rollback, representada internamente por `value_rollback`;
- o estado operacional do componente;
- a responsabilidade da aplicação de controlar o valor por meio de `value` e `onChange`.

O `value` deverá sempre representar o valor real e atual do componente, independentemente do estado em que ele se encontre.

O `value_rollback` será exclusivamente um mecanismo interno e temporário para possibilitar a restauração do valor anterior durante uma operação de alteração.

A aplicação hospedeira continuará responsável por decidir o que fazer com a fotografia produzida, inclusive quando deverá enviá-la ao backend e quando deverá permitir um novo ciclo de captura.

5.11 Gerenciamento do ciclo de vida da câmera

O `FotografoDeFACES` deverá gerenciar o acesso à câmera por meio da API `navigator.mediaDevices.getUserMedia()`, considerando que a aplicação será executada em ambiente seguro (HTTPS).

O gerenciamento da câmera deverá distinguir claramente entre **permissão, acesso ao dispositivo, existência do dispositivo e ciclo de vida do `MediaStream`**.

O componente deverá evitar solicitações repetidas de permissão sempre que a autorização da origem já tiver sido concedida e o fluxo puder reutilizar o acesso existente.

Solicitação inicial

Quando o componente precisar utilizar a câmera, deverá verificar se o acesso está disponível antes de iniciar uma nova solicitação.

Quando a permissão da origem estiver previamente concedida, o componente deverá evitar apresentar novamente uma solicitação de permissão ao usuário e deverá utilizar o acesso disponível conforme o ciclo de vida do `MediaStream`.

Quando ainda não houver autorização, o componente poderá solicitar acesso por meio de `navigator.mediaDevices.getUserMedia()`.

A solicitação deverá ocorrer somente quando houver necessidade real de utilização da câmera.

A autorização concedida pelo usuário pertence ao contexto de segurança da origem e não deverá ser tratada como uma autorização que o componente precise solicitar novamente a cada captura ou reinício de ciclo.

O componente não deverá solicitar novamente a permissão simplesmente porque:

- uma nova pessoa será fotografada;
- `restart()` foi executado;
- uma nova fotografia será capturada;
- o estado retornou para AGUARDANDO;
- uma fotografia foi confirmada;
- uma fotografia foi limpa ou substituída.

Enquanto o `MediaStream` continuar válido e em uso pelo componente, deverá ser reutilizado.

Estados relacionados ao acesso à câmera

O ciclo de acesso à câmera deverá contemplar, no mínimo, as seguintes condições:

AGUARDANDO_ACESSO



Essas condições representam o acesso ao recurso câmera e não substituem os estados funcionais da máquina de estados principal do `FotografoDeFACES`.

Acesso concedido

Quando `getUserMedia()` for concluído com sucesso, o componente deverá receber um `MediaStream` contendo a trilha de vídeo da câmera.

O componente deverá manter referência ao `MediaStream` enquanto ele for necessário e utilizá-lo para alimentar a câmera apresentada ao usuário.

A partir desse momento, a câmera deverá ser considerada disponível para o ciclo normal de operação.

O componente não deverá solicitar novamente a permissão enquanto puder reutilizar o `MediaStream` já obtido.

O estado visual poderá então prosseguir para o fluxo normal do componente, iniciando em `AGUARDANDO` e posteriormente avançando para `DETECTANDO` quando a câmera estiver efetivamente disponível para análise.

Permissão negada ou bloqueada

Quando `getUserMedia()` resultar em `NotAllowedError`, o componente deverá considerar que não possui autorização para acessar a câmera.

Essa situação poderá ocorrer porque:

- o usuário recusou a solicitação;
- o acesso foi bloqueado para a origem;
- a permissão foi revogada anteriormente;
- uma política de segurança ou `Permissions-Policy` impede o acesso;
- o contexto não possui autorização para utilizar a câmera.

O componente deverá assumir o estado de acesso:

BLOQUEADA

e deverá apresentar uma mensagem clara informando que o acesso à câmera está bloqueado.

O componente não deverá entrar em um ciclo automático de novas solicitações de permissão.

A tentativa de obter acesso novamente somente deverá ocorrer mediante uma ação apropriada do usuário ou quando o componente detectar que a permissão voltou a estar disponível.

Quando o navegador não permitir que a aplicação altere diretamente a permissão, a orientação deverá indicar ao usuário que o acesso precisa ser reativado nas configurações do navegador.

O componente não deverá tentar contornar uma decisão de bloqueio tomada pelo usuário.

Prompt ignorado ou ainda pendente

O usuário poderá simplesmente não responder ao prompt de permissão.

Nesse caso, a chamada a `getUserMedia()` poderá permanecer pendente, sem resultar imediatamente em sucesso ou erro.

O componente deverá representar essa condição como:

`SOLICITANDO_ACESSO`

e não deverá interpretar a ausência de resposta como autorização ou como recusa.

O componente também não deverá iniciar chamadas concorrentes a `getUserMedia()` enquanto uma solicitação anterior permanecer pendente.

Dispositivo inexistente ou indisponível

Quando não houver dispositivo de vídeo compatível com as restrições solicitadas, `getUserMedia()` poderá resultar em `NotFoundError`.

Nesse caso, o componente deverá assumir:

`AUSENTE`

e informar claramente que nenhuma câmera compatível foi encontrada.

Essa situação deverá ser tratada separadamente da permissão negada.

Exemplos:

- computador sem webcam;
- webcam desconectada antes da inicialização;
- câmera selecionada que deixou de existir;
- dispositivo que não atende às restrições solicitadas.

O componente não deverá solicitar repetidamente a permissão quando o problema for a inexistência do hardware.

Perda de acesso durante a utilização

Depois que a câmera estiver ativa, o acesso poderá ser perdido por razões externas ao componente.

Entre as possibilidades estão:

- desconexão física da webcam;
- falha do dispositivo;
- indisponibilidade causada pelo sistema operacional;
- outro aplicativo ou processo impedindo o acesso;
- encerramento ou interrupção da trilha de vídeo;
- erro de leitura do dispositivo.

Quando o navegador indicar `NotReadableError` durante a obtenção ou utilização do dispositivo, o componente deverá considerar que o hardware ou o ambiente de execução não está permitindo a leitura adequada da câmera.

O componente deverá interromper o ciclo de captura e apresentar uma mensagem compatível com a situação.

Quando a perda ocorrer após o `MediaStream` já ter sido obtido, o componente deverá também monitorar o encerramento ou término da `MediaStreamTrack` de vídeo e tratar a perda efetiva da câmera como indisponibilidade do recurso.

O componente não deverá permanecer indefinidamente em estado que aparente possuir uma câmera funcional quando a trilha de vídeo já tiver sido encerrada.

Recuperação após perda de acesso

Após uma perda de acesso, o componente poderá permitir uma nova tentativa de obtenção da câmera quando houver uma ação explícita do usuário ou quando a aplicação determinar que uma nova tentativa é apropriada.

A recuperação deverá respeitar novamente o estado de permissão da origem.

Se a permissão continuar concedida, uma nova chamada para obtenção do dispositivo poderá ser realizada sem que isso implique necessariamente uma nova solicitação visual de permissão ao usuário.

Se a permissão tiver sido revogada ou bloqueada, o componente deverá retornar ao tratamento de BLOQUEADA.

Se o dispositivo continuar ausente, deverá permanecer em AUSENTE.

Encerramento do acesso

Quando o `FotografoDeFACES` deixar de precisar da câmera, o componente deverá encerrar corretamente o uso do `MediaStream`.

No desmontamento do componente, todas as `MediaStreamTrack` utilizadas pelo componente deverão ser encerradas por meio de:

```
track.stop()
```

O encerramento deverá ocorrer no mecanismo de limpeza correspondente ao ciclo de vida do componente.

O objetivo é garantir que:

- a câmera deixe de ser utilizada pelo componente;
- os recursos associados ao `MediaStream` sejam liberados;
- o indicador físico de atividade da câmera seja desligado quando não houver outro consumidor utilizando o dispositivo;
- não permaneçam referências ou streams ativos desnecessariamente.

O encerramento do `MediaStream` não deverá ser confundido com a revogação da permissão concedida à origem.

A aplicação poderá desmontar o componente e, posteriormente, montá-lo novamente sem que isso signifique que o usuário precise necessariamente conceder a permissão outra vez.

Reutilização e prevenção de solicitações repetidas

O componente deverá adotar como princípio:

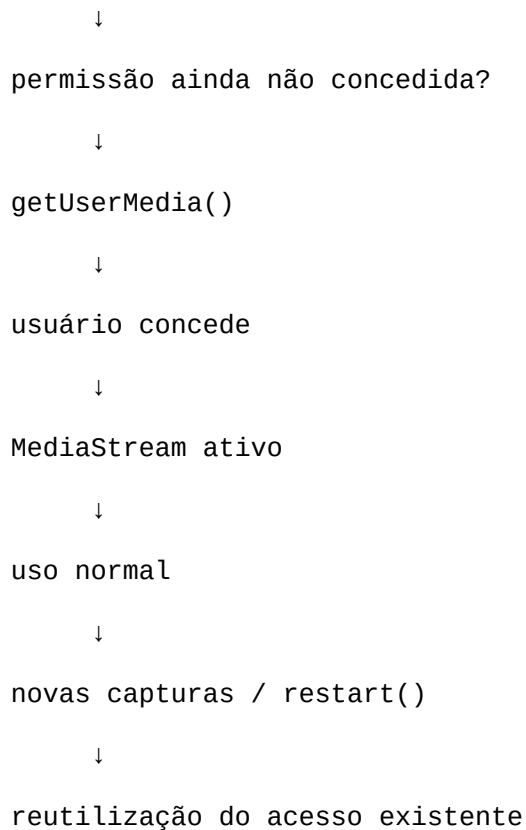
Solicitar permissão somente quando necessário; reutilizar o acesso já concedido sempre que possível; nunca solicitar novamente apenas por causa de uma nova captura.

O ciclo esperado será:

Primeiro uso

↓

verificar acesso



A desmontagem do componente deverá encerrar o `MediaStream` por razões de gerenciamento de recursos, mas isso não deverá ser interpretado como uma solicitação para que o navegador esqueça a autorização concedida à origem.

Critérios de aceite

1. Em ambiente HTTPS, o componente deverá conseguir solicitar acesso à câmera por meio de `navigator.mediaDevices.getUserMedia()`.
2. Após uma permissão concedida, o componente não deverá solicitar novamente a autorização simplesmente porque uma nova fotografia será capturada.
3. `restart()` não deverá provocar nova solicitação de permissão quando o acesso à câmera continuar disponível.
4. Uma solicitação de permissão já pendente não poderá gerar solicitações concorrentes.
5. `NotAllowedError` deverá resultar em estado de acesso BLOQUEADA.
6. `NotFoundError` deverá resultar em estado AUSENTE.
7. `NotReadableError` deverá ser tratado como falha de acesso ao dispositivo e resultar em estado de erro compatível com indisponibilidade da câmera.
8. O componente deverá detectar a perda da `MediaStreamTrack` de vídeo após a câmera ter sido iniciada.
9. O componente deverá informar ao usuário quando a câmera estiver bloqueada, ausente ou indisponível.

10. O componente não deverá tentar alterar ou contornar uma permissão negada pelo usuário.
11. Ao desmontar, o componente deverá executar `stop()` em todas as tracks que estiverem sob sua responsabilidade.
12. O encerramento das tracks deverá ocorrer independentemente de o componente estar em AGUARDANDO, DETECTANDO, CRONOMETRANDO, PRONTO ou outro estado operacional no momento do desmontamento.
13. O componente deverá diferenciar o encerramento do `MediaStream` da revogação da permissão da origem.
14. A aplicação deverá conseguir montar novamente o componente posteriormente sem que o componente solicite desnecessariamente uma nova autorização quando a permissão da origem continuar concedida.
15. O gerenciamento da câmera não deverá interferir no funcionamento normal da máquina de estados principal do `FotografoDeFaces`.

06. Máquina de estados

O `FotografoDeFaces` deverá possuir uma máquina de estados explícita, responsável por controlar o ciclo de captura e impedir que ações ocorram fora do momento apropriado.

A máquina de estados deverá ser a referência central para o comportamento operacional do componente. A interface visual, os comandos disponibilizados pelo `useRef`, os botões internos, os eventos e os processos de captura deverão respeitar o estado atual e o valor atual de `value`.

A máquina deverá reagir às alterações de `value` recebidas da aplicação, mantendo coerência entre o valor atual do componente e seu estado operacional.

6.1 Estados

A máquina de estados será composta pelos seguintes estados:

AGUARDANDO

Estado inicial e estado de retorno do componente.

Representa o momento em que o `FotografoDeFaces` não possui uma fotografia atualmente disponível em `value` e está disponível para iniciar um novo processo de identificação de uma face candidata.

No modo quiosque, somente neste estado poderá ser selecionada uma nova pessoa como candidata.

DETECTANDO

Estado em que o componente está procurando uma ou mais faces que possam participar do processo de captura.

A existência de uma face detectada ainda não significa que ela esteja apta para captura.

No modo quiosque, poderão existir várias faces simultaneamente, sendo responsabilidade do componente selecionar uma única candidata conforme os critérios definidos para esse modo.

AVALIANDO

Estado em que o componente possui uma candidata e está verificando continuamente se ela atende aos critérios necessários para uma fotografia facial adequada.

Os critérios poderão envolver, entre outros:

- quantidade de faces;
- enquadramento;
- distância;
- pose;
- nitidez;
- iluminação;
- estabilidade;
- posicionamento;
- demais critérios de qualidade definidos pelo componente.

No modo quiosque, a candidata deverá permanecer protegida pelo mecanismo de Face Lock.

PRONTO

Estado que representa a condição em que a candidata atual atende aos critérios necessários para que uma fotografia possa ser produzida.

Somente neste estado uma captura poderá ser efetivamente disparada.

O estado PRONTO não significa que uma fotografia já tenha sido produzida. Significa que a imagem atualmente disponível na câmera está apta para ser capturada.

CRONOMETRANDO

Estado utilizado quando `autoCaptureAfter` possuir valor superior a zero.

Representa o período de espera entre a identificação da condição PRONTO e o disparo automático.

Durante esse período, o componente deverá continuar monitorando a qualidade e a estabilidade da candidata.

O cronômetro deverá ser apresentado visualmente pelo componente quando aplicável, e suas informações também deverão estar disponíveis por meio do `useRef`.

Caso a candidata deixe de atender aos critérios necessários durante a contagem, o disparo não deverá ocorrer e o componente deverá retornar a `AGUARDANDO`, iniciando posteriormente um novo ciclo.

CAPTURANDO

Estado transitório em que o componente está efetivamente produzindo a fotografia.

Nesse estado, o disparo já foi autorizado porque as condições necessárias foram atendidas.

O componente deverá realizar a captura e o tratamento da imagem, incluindo os procedimentos necessários para produzir a fotografia final.

Nenhum novo disparo deverá ser aceito enquanto a operação estiver em andamento.

FOTOGRAFIA_PRONTA

Estado em que o componente possui uma fotografia atualmente disponível em `value` e a apresenta visualmente ao usuário.

A fotografia poderá ter sido:

- fornecida inicialmente pela aplicação;
- produzida por uma captura realizada pelo componente;
- fornecida pela aplicação após uma alteração;
- restaurada após uma operação de cancelamento.

O estado `FOTOGRAFIA_PRONTA` representa, portanto, a existência de um valor de fotografia atualmente disponível no componente, e não exclusivamente uma fotografia que tenha sido previamente confirmada.

Quando `reviewFor` estiver habilitado, o componente poderá disponibilizar as ações de alteração previstas para esse estado.

ERRO

Estado destinado às situações em que uma operação de captura ou tratamento não consiga produzir uma fotografia válida.

O erro deverá ser informado visualmente ao usuário e também deverá estar disponível para consulta pela aplicação.

O componente deverá evitar permanecer indefinidamente nesse estado. Conforme a natureza do erro, poderá retornar a `AGUARDANDO` ou aguardar uma ação explícita da aplicação.

6.2 Relação entre `value` e a máquina de estados

O `value` deverá representar sempre o valor atual do componente.

A máquina de estados deverá reagir ao valor recebido pela aplicação:

```
value = null
```

↓

AGUARDANDO

Quando `value` passar a representar uma fotografia válida:

```
value = fotografia
```

↓

FOTOGRAFIA_PRONTA

Essa regra também deverá ser aplicada quando uma fotografia for fornecida inicialmente pela aplicação.

O componente não deverá considerar que `value` representa exclusivamente a última fotografia confirmada.

Durante uma operação de alteração, também será válida a situação:

```
value = nova fotografia
```

```
value_rollback = fotografia anterior
```

Nesse caso, o estado continuará sendo:

FOTOGRAFIA_PRONTA

O `value_rollback` será utilizado exclusivamente para permitir o cancelamento da operação e a restauração do valor anterior.

6.3 Transições principais

Quando o componente estiver sem fotografia em `value`, o fluxo normal de captura deverá seguir:

AGUARDANDO

↓

DETECTANDO



AVALIANDO



PRONTO



CAPTURANDO



FOTOGRAFIA_PRONTA

Quando houver captura automática temporizada:

AGUARDANDO



DETECTANDO



AVALIANDO



PRONTO



CRONOMETRANDO



CAPTURANDO



FOTOGRAFIA_PRONTA

O estado PRONTO será, portanto, o ponto central de autorização da captura.

6.4 Perda da candidata

A máquina deverá ser capaz de retornar ao início de um novo ciclo quando a candidata deixar de atender às condições necessárias.

Se, durante AVALIANDO, a candidata perder efetivamente as condições necessárias para continuidade:

AVALIANDO

↓

AGUARDANDO

↓

DETECTANDO

O mesmo deverá ocorrer durante PRONTO ou CRONOMETRANDO, quando a condição de prontidão deixar de existir de maneira suficiente para invalidar o disparo:

PRONTO

↓

AGUARDANDO

↓

DETECTANDO

ou:

CRONOMETRANDO

↓

AGUARDANDO

↓

DETECTANDO

O retorno a AGUARDANDO é deliberado. O componente deverá iniciar um novo processo de busca, permitindo que uma nova pessoa possa se tornar candidata.

No modo quiosque, esse comportamento é especialmente importante: a perda da candidata protegida pelo Face Lock não deverá fazer com que outra face assuma automaticamente o foco do ciclo atual.

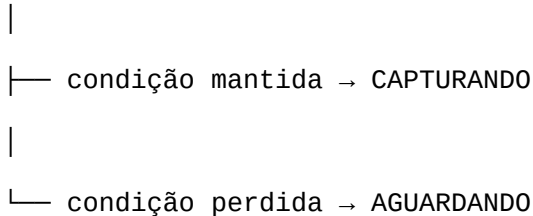
Somente após o retorno a AGUARDANDO poderá ocorrer uma nova seleção de candidato.

6.5 Cancelamento do disparo automático

O estado CRONOMETRANDO somente poderá avançar para CAPTURANDO se, ao final da contagem, a candidata continuar atendendo aos critérios necessários.

Assim:

CRONOMETRANDO



↓

DETECTANDO

O componente não deverá capturar uma fotografia apenas porque o cronômetro terminou.

O término do cronômetro é uma condição necessária, mas não suficiente. A candidata deverá continuar apta no momento do disparo.

6.6 Captura manual

No modo manual, o botão de captura deverá ser disponibilizado pelo próprio componente somente quando o estado for:

PRONTO

O usuário poderá então disparar a fotografia pelo botão apresentado na interface.

A aplicação também poderá realizar o mesmo disparo externamente por meio do `useRef`.

Em ambos os casos, a regra será a mesma:

Não existe captura válida fora de PRONTO.

6.7 FOTOGRAFIA_PRONTA e revisão

Quando o componente estiver em FOTOGRAFIA_PRONTA e `reviewFor` estiver habilitado, as ações de revisão deverão respeitar o fluxo definido para o valor atual.

Quando não houver `value_rollback`, poderão ser disponibilizadas:

[Trocar] [Limpar]

Quando houver uma operação de alteração em andamento e `value_rollback` contiver o valor anterior, poderão ser disponibilizadas:

[Confirmar] [Cancelar]

É possível, portanto, que durante uma operação de alteração existam simultaneamente:

`value = fotografia atual`

`value_rollback = fotografia anterior`

Essa combinação é válida e não constitui um estado adicional da máquina.

6.8 Reinício do ciclo

O comando público de reinício deverá conduzir o componente novamente ao estado:

AGUARDANDO

Esse retorno deverá representar um novo ciclo de busca.

Essa regra é particularmente importante no modo quiosque: após o processamento de uma pessoa, o componente não deverá simplesmente continuar avaliando as faces existentes. Ao retornar a AGUARDANDO, deverá iniciar uma nova busca e permitir que outra pessoa se torne candidata.

6.9 Regra fundamental da máquina

A máquina de estados deverá garantir que:

Nenhuma fotografia poderá ser capturada enquanto o componente não estiver no estado PRONTO.

Essa regra deverá ser respeitada tanto pelo botão interno de captura quanto pelos comandos disponibilizados pelo `useRef` e pelos mecanismos automáticos de disparo.

Ao mesmo tempo:

O estado da máquina deverá refletir as condições atuais do componente e o `value` recebido pela aplicação.

O `FotografoDeFACES` deverá, portanto, manter o controle do momento correto da captura sem assumir para si o controle do valor público do componente.

07. Comportamento de cada estado

Cada estado do `FotografoDeFACES` deverá possuir comportamento claramente definido, tanto internamente quanto na interface apresentada ao usuário.

O componente deverá manter coerência entre:

- o estado interno da máquina;
- a imagem apresentada;
- as mensagens de orientação;
- os indicadores visuais;
- os comandos disponíveis;
- as informações disponibilizadas pelo `useRef`;
- os eventos emitidos para a aplicação hospedeira.

7.1 AGUARDANDO

Representa o estado inicial e o ponto de retorno para um novo ciclo de captura.

Neste estado:

- não haverá uma candidata selecionada;
- o `Face Lock` estará inativo;
- nenhuma captura estará em andamento;
- o componente poderá apresentar a câmera ao vivo;
- o componente estará preparado para iniciar uma nova busca;
- no modo quiosque, uma nova pessoa poderá tornar-se candidata.

Não deverá haver moldura de face quando não existir uma face detectada.

A mensagem poderá orientar o usuário, por exemplo:

"Aguardando próxima pessoa."

Quando a análise da câmera estiver disponível, o componente poderá avançar para `DETECTANDO`.

7.2 DETECTANDO

Neste estado, o componente estará procurando faces que possam participar do processo de captura.

Quando uma ou mais faces forem detectadas, o componente deverá apresentar uma moldura amarela sobre cada face identificada, desde que a exibição da moldura esteja habilitada.

A moldura amarela representa visualmente que uma face foi detectada, mas ainda não significa que ela esteja apta para captura.

Autorretrato e assistido

Nesses modos, a condição necessária será a existência de exatamente uma face detectada.

- nenhuma face → permanece procurando;
- exatamente uma face → poderá avançar para AVALIANDO;
- mais de uma face → não poderá avançar para AVALIANDO.

Quiosque

No modo quiosque, poderão existir várias faces simultaneamente no campo de visão.

Nesse caso:

- todas as faces detectadas poderão receber molduras amarelas;
- o componente deverá avaliar as faces disponíveis;
- deverá selecionar uma única candidata de acordo com os critérios de qualidade definidos para o modo;
- após a seleção, deverá aplicar o Face Lock;
- somente a candidata selecionada poderá prosseguir no ciclo.

A existência de várias molduras amarelas no quiosque será, portanto, uma situação normal.

Mensagem possível:

"Posicione-se diante da câmera."

7.3 AVALIANDO

Neste estado existe uma candidata sendo analisada continuamente.

A candidata deverá continuar representada por uma moldura amarela enquanto ainda não atender a todos os critérios necessários.

O componente deverá verificar, entre outros:

- enquadramento;
- distância;
- pose;
- nitidez;
- iluminação;
- estabilidade;
- posicionamento;
- quantidade de faces, conforme o modo de operação.

O componente deverá indicar ao usuário a principal condição que precisa ser corrigida, por exemplo:

"Aproxime-se."

ou:

"Mova-se um pouco para a direita."

O componente deverá evitar apresentar várias instruções simultaneamente quando uma única orientação for suficiente.

Quando todos os critérios forem atendidos, a candidata será considerada apta e o componente avançará para PRONTO.

7.4 PRONTO

Este estado representa uma condição fundamental do componente:

Existe uma face candidata identificável e estável, suficientemente enquadrada, nítida, iluminada e frontal, dentro dos limites definidos pelo componente para produzir uma fotografia facial adequada ao processamento posterior.

Neste estado:

- a candidata deverá continuar sendo monitorada;
- o Face Lock permanecerá ativo quando aplicável;
- a moldura da candidata deverá ficar verde, desde que sua exibição esteja habilitada;
- a fotografia ainda não terá sido produzida;
- o componente poderá disponibilizar o disparo manual;
- o disparo automático poderá ser iniciado conforme `autoCaptureAfter`.

Mensagem padrão possível:

"Captura OK."

Se `autoCaptureAfter` for `null`, o componente permanecerá em PRONTO aguardando o disparo manual.

Se `autoCaptureAfter` for 0, deverá iniciar imediatamente o processo de captura.

Se `autoCaptureAfter` for maior que 0, deverá avançar para CRONOMETRANDO.

7.5 CRONOMETRANDO

Este estado existirá somente quando houver captura automática temporizada.

O componente deverá apresentar visualmente o andamento da contagem.

Por exemplo:

"Aguarde... 1 de 3 segundos."

"Aguarde... 2 de 3 segundos."

"Aguarde... 3 de 3 segundos."

O tempo transcorrido, o tempo restante e as demais informações relevantes do cronômetro deverão estar disponíveis para consulta pela aplicação por meio do `useRef`.

Durante toda a contagem, o componente deverá continuar avaliando a candidata.

A moldura poderá permanecer verde, indicando que a condição de prontidão continua válida, enquanto o cronômetro funcionará como informação adicional.

Caso a candidata seja efetivamente perdida:

CRONOMETRANDO

↓

AGUARDANDO

↓

DETECTANDO

O componente não deverá efetuar o disparo.

Caso a candidata permaneça apta até o final da contagem:

CRONOMETRANDO

↓

CAPTURANDO

O término da contagem, portanto, não deverá substituir a necessidade de a candidata continuar apta para captura.

7.6 CAPTURANDO

Representa o momento em que o componente está efetivamente produzindo a fotografia.

Durante esse estado:

- o disparo deverá ficar bloqueado;
- um novo disparo não deverá ser aceito;
- o componente deverá executar o processo de captura;

- deverá selecionar a imagem correspondente à candidata;
- deverá realizar o recorte;
- deverá realizar o redimensionamento;
- deverá realizar os tratamentos necessários;
- deverá gerar a fotografia final no formato estabelecido.

A interface poderá apresentar uma pequena animação de captura.

A moldura poderá apresentar uma animação breve para proporcionar ao usuário uma percepção clara de que a fotografia foi efetivamente registrada.

Se a fotografia for produzida corretamente:

CAPTURANDO

↓

FOTOGRAFIA_PRONTA

Se houver falha:

CAPTURANDO

↓

ERRO

7.7 FOTOGRAFIA_PRONTA

Neste estado, o componente deverá apresentar como conteúdo principal a fotografia atualmente representada por `va.lue`.

A fotografia poderá ter sido:

- fornecida inicialmente pela aplicação;
- produzida por uma captura;
- fornecida pela aplicação após uma alteração;
- restaurada após um cancelamento.

A fotografia deverá ser apresentada com moldura azul, diferenciando visualmente uma fotografia já disponível no componente de uma face que ainda está sendo detectada ou avaliada.

Sem possibilidade de revisão

Quando `reviewFor` não estiver habilitado, o componente deverá permanecer apresentando a fotografia atualmente representada por `va.lue`, seguindo o fluxo padrão definido para o componente.

Com possibilidade de revisão

Quando `reviewFor` estiver habilitado, o componente poderá disponibilizar as ações de alteração.

Quando não existir uma operação de alteração em andamento, deverão ser apresentadas, conforme a configuração visual do componente:

[Trocar] [Limpar]

Essas ações não deverão ser consideradas estados adicionais da máquina.

Trocar

Ao selecionar Trocar:

1. o componente deverá preservar internamente o `value` atual em `value_rollback`;
2. deverá solicitar a alteração do `value` para `null` por meio de `onChange`;
3. deverá retornar ao fluxo normal de captura;
4. deverá permitir que uma nova fotografia seja produzida.

Durante esse processo, poderão existir:

`value = null`

`value_rollback = fotografia anterior`

Quando uma nova fotografia for produzida, o componente deverá solicitar sua atribuição por meio de `onChange`.

Nesse momento, poderão existir:

`value = nova fotografia`

`value_rollback = fotografia anterior`

Essa situação é válida e representa uma alteração em andamento.

O componente deverá então apresentar:

[Confirmar] [Cancelar]

Limpar

Ao selecionar Limpar:

5. o componente deverá preservar internamente o `value` atual em `value_rollback`;
6. deverá solicitar `onChange(null)`;
7. deverá retornar ao estado AGUARDANDO;
8. deverá permitir um novo processo de captura.

Nesse momento:

```
value = null
```

```
value_rollback = fotografia anterior
```

As opções:

```
[ Confirmar ] [ Cancelar ]
```

deverão ser disponibilizadas para a operação de alteração, conforme as regras de `reviewFor`.

Confirmar

Confirmar deverá consolidar o `value` atualmente exibido como o novo valor do componente.

Nesse momento:

```
value_rollback = null
```

O `value` atual permanecerá inalterado.

Se a operação tiver sido iniciada por Trocar, a nova fotografia passará a ser o valor atual do componente.

Se a operação tiver sido iniciada por Limpar, o valor atual permanecerá `null`.

Após a confirmação, poderão voltar a ser disponibilizadas:

```
[ Trocar ] [ Limpar ]
```

conforme a configuração do componente.

Cancelar

Cancelar deverá abandonar a alteração em andamento.

O componente deverá solicitar por meio de `onChange` a restauração do valor armazenado em `value_rollback`.

Após a aplicação atualizar `value`, o componente deverá limpar a referência interna:

```
value_rollback = null
```

Assim, se:

```
value = nova fotografia
```

```
value_rollback = fotografia anterior
```

o cancelamento produzirá:

```
onChange(fotografia anterior)
```

e, após a atualização:

```
value = fotografia anterior
```

```
value_rollback = null
```

Se a operação tiver sido iniciada quando não havia fotografia anterior:

```
value = nova fotografia
```

```
value_rollback = null
```

o cancelamento deverá restaurar:

```
value = null
```

Janela temporal de revisão

Quando `reviewFor` estiver definido com valor maior que zero, o tempo será utilizado como janela para as ações disponíveis após uma alteração.

A mesma regra temporal deverá ser aplicada tanto às opções:

```
[ Trocar ] [ Limpar ]
```


quanto às opções:

[Confirmar] [Cancelar]

Quando o tempo terminar, as opções deverão desaparecer.

Se a janela terminar enquanto Confirmar e Cancelar estiverem disponíveis, deverá prevalecer automaticamente o equivalente a Confirmar.

Após o encerramento da janela, não haverá mais ação de troca, limpeza, confirmação ou cancelamento disponível para aquela operação.

Se reviewFor for null, não haverá janela temporal de revisão.

7.8 ERRO

Representa uma falha ocorrida durante a captura ou o processamento da fotografia.

O componente deverá:

- impedir que uma fotografia inválida seja considerada produzida com sucesso;
- informar visualmente o problema;
- disponibilizar informações do erro à aplicação;
- manter o estado consistente;
- impedir novos disparos enquanto a operação estiver impossibilitada.

A moldura da candidata poderá ser apresentada em vermelho quando o erro estiver diretamente relacionado à captura ou à impossibilidade de produzir uma fotografia válida.

Mensagem possível:

"Não foi possível realizar a captura. Tente novamente."

A aplicação poderá utilizar o useRef para consultar o erro e decidir se deverá solicitar um novo ciclo.

Conforme a natureza da falha, o componente poderá retornar para:

ERRO → AGUARDANDO

ou permanecer em ERRO até receber uma ação explícita.

7.9 Comportamento visual das molduras

Quando a funcionalidade de moldura das faces estiver habilitada, o componente deverá utilizar a cor como comunicação visual do processo:

Situação	Moldura
Nenhuma face detectada	Não exibir
Face(s) detectada(s) em DETECTANDO	Amarela
Face candidata sendo avaliada	Amarela
Candidata aprovada em PRONTO	Verde
Captura em andamento	Verde + animação, se adotada
Fotografia produzida	Azul
Falha na captura	Vermelha

No modo quiosque, poderá haver várias molduras amarelas simultaneamente. Depois que uma candidata for selecionada e protegida pelo Face Lock, somente ela poderá avançar para PRONTO e apresentar a moldura verde.

A cor não deverá ser configurável pela aplicação. Ela fará parte da linguagem visual e do comportamento padrão do `FotografoDeFaces`.

A espessura e a possibilidade de exibição da moldura poderão ser configuradas pelas propriedades do componente.

7.10 Coerência entre interface e useRef

O componente deverá fornecer informações equivalentes tanto pela interface visual quanto pelo `useRef`.

Por exemplo, durante `CRONOMETRANDO`, o componente poderá apresentar:

"Aguarde... 2 de 3 segundos."

Enquanto a aplicação poderá consultar pelo `useRef` informações equivalentes sobre:

- estado atual;
- etapa do cronômetro;
- tempo transcorrido;
- tempo restante;
- mensagem atual;
- qualidade da candidata.

Isso permitirá que a aplicação hospedeira escolha entre utilizar a interface própria do componente ou construir uma interface externa utilizando as informações disponibilizadas pelo `useRef`.

O mesmo princípio deverá ser aplicado às demais situações relevantes do ciclo.

7.11 Regra de consistência

Nenhuma informação visual deverá indicar que uma fotografia está pronta para captura quando a máquina de estados não estiver efetivamente em `PRONTO`.

Da mesma forma:

- a moldura amarela deverá representar uma face detectada ou em avaliação;
- a moldura verde deverá representar uma candidata apta para captura;
- a moldura azul deverá representar uma fotografia produzida;
- a moldura vermelha poderá representar uma falha de captura;
- o botão de captura deverá estar disponível somente quando o disparo for permitido;
- capturar () deverá respeitar a máquina de estados;
- o cronômetro somente deverá existir durante CRONOMETRANDO;
- Trocar e Limpar deverão iniciar operações de alteração sem consolidá-las imediatamente;
- Confirmar deverá consolidar o value atualmente representado pelo componente;
- Cancelar deverá restaurar o valor armazenado em value_rollback;
- value deverá sempre representar o valor atual do componente.

O objetivo é evitar divergência entre o que o usuário vê, o que a aplicação consulta e o que o componente efetivamente permite executar.

08. Face Lock

O Face Lock é o mecanismo responsável por manter uma única pessoa como candidata durante um ciclo de captura, impedindo que o componente troque de alvo enquanto está avaliando ou preparando a fotografia.

O Face Lock não representa a fotografia produzida e não está relacionado ao value.

Ele atua exclusivamente sobre a face candidata que está sendo acompanhada pela câmera antes da captura.

8.1 Objetivo

O objetivo do Face Lock é garantir que, uma vez selecionada uma candidata, o FotografoDeFACES continue trabalhando com aquela mesma pessoa até que o ciclo seja encerrado ou a candidata seja considerada perdida.

Isso é especialmente importante no modo quiosque, em que várias pessoas poderão aparecer simultaneamente diante da câmera.

Sem Face Lock, uma pessoa poderia ser selecionada inicialmente e, durante a avaliação, outra pessoa entrar no campo de visão e apresentar melhores condições momentâneas.

Nesse caso, o componente poderia trocar indevidamente de alvo.

O Face Lock deverá impedir esse comportamento.

8.2 Ativação

O Face Lock será ativado quando o componente selecionar uma face candidata válida para o ciclo.

A partir desse momento, essa face passará a ser o único alvo do processo.

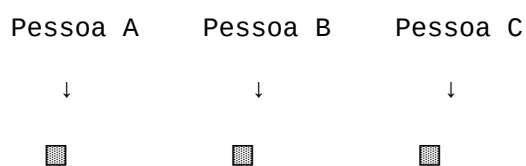
O componente deverá acompanhar continuamente essa candidata utilizando as informações disponíveis para identificação e acompanhamento da mesma face ao longo dos frames.

A presença de outras faces não deverá provocar automaticamente a substituição da candidata bloqueada.

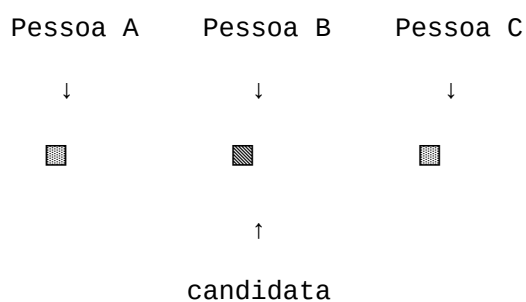
8.3 Comportamento no modo quiosque

O Face Lock terá importância especial no modo quiosque.

O cenário poderá ser:



O componente poderá avaliar as três faces e selecionar, conforme seus critérios, a Pessoa B como candidata:



A partir desse momento, a Pessoa B será o alvo bloqueado.

Se a Pessoa A ou a Pessoa C se aproximarem mais da câmera, apresentarem melhor iluminação ou passarem a ter uma qualidade momentaneamente superior, isso não deverá fazer o componente trocar automaticamente de candidato.

O Face Lock deverá preservar a Pessoa B como alvo.

8.4 Acompanhamento da candidata

Enquanto o Face Lock estiver ativo, o componente deverá acompanhar a candidata continuamente.

A identificação da mesma face poderá utilizar informações como:

- posição;
- tamanho;
- proporções do enquadramento;
- trajetória entre frames;
- características geométricas disponíveis pelo mecanismo de detecção;
- demais informações necessárias para diferenciar a candidata das outras faces.

O componente não deverá realizar reconhecimento de identidade.

O objetivo é apenas determinar que a face observada no frame atual corresponde à mesma candidata que foi selecionada para aquele ciclo.

8.5 Outras faces durante o Face Lock

A presença de outras pessoas não deverá interromper o ciclo.

Por exemplo:

Face Lock → Pessoa B

Pessoa A entra → ignorar

Pessoa C aproxima → ignorar

Pessoa A sai → ignorar

Pessoa C sai → ignorar

Pessoa B permanece → continuar avaliando

As outras faces poderão continuar sendo visualmente indicadas por molduras amarelas quando a funcionalidade de moldura estiver habilitada, mas não poderão assumir o estado PRONTO enquanto o Face Lock estiver mantido sobre outra candidata.

Apenas a candidata bloqueada poderá apresentar a condição visual verde correspondente a PRONTO.

8.6 Perda da candidata

O Face Lock não deverá obrigar o componente a manter indefinidamente uma candidata que deixou de ser acompanhável.

Se a face bloqueada desaparecer ou deixar de ser identificável por um período suficiente para caracterizar perda da candidata, o componente deverá considerar que o Face Lock foi perdido.

Entretanto, uma perda momentânea de um frame não deverá necessariamente encerrar o ciclo.

O componente deverá possuir tolerância temporal para pequenas falhas de detecção, evitando que oscilações naturais da câmera ou do algoritmo provoquem reinícios desnecessários.

Somente quando a candidata for considerada efetivamente perdida o ciclo deverá ser interrompido.

8.7 Perda durante avaliação ou cronômetro

Se a candidata for perdida enquanto estiver em:

AVALIANDO

ou:

CRONOMETRANDO

o disparo não deverá ocorrer.

O componente deverá abandonar a condição de prontidão, liberar o Face Lock e retornar ao estado:

AGUARDANDO

A partir daí deverá iniciar um novo ciclo:

AGUARDANDO

↓

DETECTANDO

Uma nova candidata somente poderá ser selecionada nesse novo processo de detecção.

Essa regra é deliberada: o componente não deverá aproveitar diretamente a avaliação de outra face existente no mesmo ciclo.

8.8 Face Lock durante CAPTURANDO

Ao entrar em CAPTURANDO, a candidata já deverá estar definida e estabilizada.

O componente deverá utilizar essa candidata como referência para produzir a fotografia.

A entrada de outras faces no campo de visão durante o processamento não deverá alterar a fotografia que está sendo produzida.

O Face Lock deverá, portanto, permanecer logicamente associado à candidata até que a captura seja concluída ou falhe.

8.9 Face Lock e FOTOGRAFIA_PRONTA

Após a fotografia ser produzida e o componente entrar em:

FOTOGRAFIA_PRONTA

o Face Lock da captura anterior deixará de ser relevante.

A fotografia atualmente representada por `value` passará a ser o objeto visual principal do componente.

O componente deverá permanecer apresentando essa fotografia até que:

- uma operação de Trocar seja iniciada;
- a fotografia seja Limpar;
- a aplicação altere o `value`;
- uma operação de Cancelar restaure um valor anterior;
- ou o ciclo seja reiniciado.

O Face Lock não deverá ser utilizado para controlar a fotografia apresentada nem deverá ser confundido com `value_rollback`.

8.10 Liberação do Face Lock

O Face Lock deverá ser liberado quando o ciclo deixar de estar vinculado à candidata atual.

Isso ocorrerá, principalmente, quando:

- a candidata for efetivamente perdida;
- o processo retornar a AGUARDANDO;
- a aplicação solicitar `restart()`;
- o ciclo atual for definitivamente encerrado.

Ao retornar a:

AGUARDANDO

não deverá existir qualquer vínculo com a pessoa anteriormente selecionada.

Isso é fundamental no modo quiosque.

A pessoa que será capturada no próximo ciclo deverá ser escolhida novamente pelo mecanismo de seleção de candidatos.

8.10 Critérios de qualidade da face

A avaliação de qualidade deverá ser utilizada pelo `FotografoDeFACES` para determinar se uma face detectada possui condições suficientes para produzir uma fotografia facial adequada ao processamento posterior.

A qualidade deverá ser utilizada em dois níveis distintos:

1. **critérios obrigatórios**, que determinam se a face pode ser considerada apta para o estado PRONTO;
2. **critérios graduais**, utilizados para comparar e selecionar a melhor candidata quando houver mais de uma face possível, especialmente no modo quiosque.

A avaliação de qualidade não deverá substituir os requisitos de estabilidade e enquadramento definidos para o ciclo de captura.

Critérios obrigatórios para PRONTO

Uma face somente poderá ser considerada apta ao estado PRONTO quando atender simultaneamente aos critérios mínimos definidos pelo componente.

A candidata deverá:

- ser identificável pelo mecanismo de detecção facial;
- estar suficientemente enquadrada dentro dos limites estabelecidos pelo componente;
- apresentar posicionamento compatível com a captura facial;
- possuir nitidez suficiente para produzir uma fotografia utilizável;
- apresentar iluminação suficiente para permitir a captura adequada;
- apresentar orientação frontal compatível com a finalidade biométrica;
- permanecer estável durante o período definido para confirmação da candidata.

A condição poderá ser resumida como:

Face identificável

+

Enquadramento adequado

+

Nitidez suficiente

+

Iluminação suficiente

+

Orientação frontal adequada

+

Estabilidade

↓

Candidata apta

↓

PRONTO

O atendimento parcial desses critérios não deverá ser suficiente para determinar PRONTO.

Uma face que esteja detectada, mas não atenda aos critérios mínimos de qualidade, deverá permanecer em avaliação ou retornar ao ciclo de detecção conforme as regras da máquina de estados.

Critérios graduais de qualidade

Além dos critérios mínimos obrigatórios, o componente poderá avaliar características graduais da face para determinar qual candidata apresenta melhores condições para a fotografia final.

Entre os critérios considerados poderão estar:

- nitidez;
- iluminação;
- frontalidade;
- tamanho aparente da face;
- centralidade e posicionamento;
- proporção da face dentro da área de captura;
- estabilidade da detecção;
- qualidade geral da imagem facial.

Esses critérios não deverão ser tratados exclusivamente como condições binárias.

Por exemplo, duas faces podem atender aos critérios mínimos para captura, mas uma delas poderá apresentar maior nitidez, melhor iluminação e melhor frontalidade. Nesse caso, a candidata com melhor qualidade deverá receber maior prioridade na seleção.

A avaliação deverá produzir uma classificação relativa entre as candidatas, sem alterar os critérios mínimos obrigatórios para que uma face seja considerada apta.

Relação entre qualidade e Face Lock

O Face Lock deverá garantir a estabilidade da candidata durante o ciclo de captura.

A qualidade deverá avaliar se essa candidata possui características suficientes para produzir uma fotografia adequada.

Portanto:

DETECTANDO

↓

Faces detectadas

↓

Identificação das candidatas

↓

Avaliação de qualidade

↓

Seleção / priorização

↓

Face Lock

↓

Estabilidade confirmada

↓

PRONTO

A perda da estabilidade da candidata deverá ser tratada conforme as regras estabelecidas para o Face Lock, não devendo ser mascarada por uma pontuação elevada de qualidade.

Qualidade no modo quiosque

Quando houver mais de uma face detectada no modo quiosque, a qualidade deverá ser utilizada para selecionar a melhor candidata.

A seleção deverá considerar somente faces que atendam aos critérios mínimos necessários para captura.

Uma face que não seja suficientemente adequada não deverá ser selecionada apenas por apresentar uma pontuação superior em algum critério isolado.

O objetivo da seleção será encontrar a face que apresente o melhor conjunto de características para produzir a fotografia facial final.

A seleção da melhor face será detalhada na Parte 09 — Seleção da melhor face no quiosque.

Qualidade no modo autorretrato e assistido

Nos modos autorretrato e assistido, o componente deverá utilizar os critérios mínimos de qualidade para determinar se a face detectada está apta a prosseguir para PRONTO.

Quando houver mais de uma face detectada nesses modos, deverá ser aplicada a regra específica definida para cada modo, sem utilizar automaticamente a lógica de seleção de melhor face estabelecida para o quiosque.

Princípio geral

A qualidade deverá ser entendida como:

A capacidade da candidata de produzir uma fotografia facial adequada ao processamento posterior, considerando enquadramento, nitidez, iluminação, orientação, estabilidade e demais critérios relevantes definidos pelo componente.

O componente não deverá realizar reconhecimento biométrico nessa etapa.

A avaliação de qualidade deverá determinar apenas se a imagem apresenta condições técnicas suficientes para ser capturada e disponibilizada como fotografia final.

Critérios de aceite

1. Uma face somente poderá chegar a PRONTO quando atender simultaneamente aos critérios obrigatórios.
2. A simples detecção de uma face não deverá ser suficiente para determinar PRONTO.
3. A qualidade deverá considerar, no mínimo, enquadramento, nitidez, iluminação, orientação frontal e estabilidade.

4. Os critérios obrigatórios deverão ser distintos dos critérios graduais utilizados para ranqueamento.
5. Uma candidata que não atenda aos requisitos mínimos não poderá ser selecionada como fotografia final apenas por apresentar vantagem em um critério isolado.
6. Quando houver múltiplas candidatas válidas no modo quiosque, os critérios graduais deverão contribuir para a seleção da melhor candidata.
7. O Face Lock deverá continuar sendo responsável pela estabilidade da candidata, não sendo substituído pela pontuação de qualidade.
8. A avaliação de qualidade não deverá realizar reconhecimento ou comparação biométrica.
9. Os critérios de qualidade deverão ser aplicados de forma consistente durante o ciclo de detecção e captura.
10. A fotografia somente poderá ser produzida após a candidata cumprir os requisitos necessários para alcançar PRONTO.

8.11 Regra fundamental

O Face Lock deverá obedecer à seguinte regra:

Depois que uma candidata for selecionada, o FotografoDeFACES não poderá substituí-la por outra pessoa durante o mesmo ciclo de captura.

A troca de candidato somente poderá ocorrer depois que o Face Lock for liberado e o componente retornar a AGUARDANDO, iniciando um novo processo de detecção.

Assim, no modo quiosque, mesmo que várias pessoas estejam diante da câmera, somente uma pessoa por vez poderá ser o alvo efetivo do processo de captura.

A regra também garante que uma pessoa que entre no enquadramento durante um ciclo não possa assumir o processo simplesmente por apresentar melhores condições momentâneas.

09. Seleção da melhor face no quiosque

No modo **quiosque**, o FotografoDeFACES deverá considerar que pode haver mais de uma pessoa simultaneamente diante da câmera.

O componente não deverá exigir que exista apenas uma face visível para iniciar a avaliação. Em vez disso, deverá detectar as faces presentes e determinar qual delas apresenta as melhores condições para se tornar a candidata do ciclo de captura.

A seleção deverá ser baseada principalmente na **qualidade da face para uma fotografia biométrica**, e não simplesmente na distância da pessoa em relação à câmera.

9.1 Detecção de múltiplas faces

Enquanto estiver em DETECTANDO, o componente poderá identificar:

- nenhuma face;
- uma única face;
- várias faces.

Quando houver várias faces, cada uma deverá ser tratada como uma possível candidata.

As faces detectadas deverão ser apresentadas com a **moldura amarela**, quando a exibição da moldura estiver habilitada.

A moldura amarela representa uma face detectada que ainda não foi selecionada como candidata PRONTA.

9.2 Critérios para seleção

A escolha da candidata deverá considerar os mesmos requisitos fundamentais utilizados para determinar se uma fotografia possui qualidade suficiente.

Entre os principais critérios estarão:

- qualidade do enquadramento;
- posição da face em relação à região de captura;
- tamanho adequado da face;
- distância aproximada da câmera;
- nitidez;
- iluminação;
- pose;
- estabilidade;
- possibilidade de acompanhamento consistente da face entre os frames.

A proximidade da câmera poderá ser utilizada como uma informação complementar, mas **não deverá ser o critério principal de seleção**.

Uma pessoa mais próxima da câmera não deverá necessariamente ser escolhida se outra pessoa apresentar condições significativamente melhores para uma fotografia facial adequada.

9.3 Qualidade como critério de escolha

Quando houver várias faces, o componente deverá avaliar a qualidade de cada candidata e selecionar aquela que apresentar o melhor conjunto de condições para a captura.

Conceitualmente:

Face A → qualidade 72%

Face B → qualidade 91%

Face C → qualidade 64%

↓

Face B selecionada

Os valores acima representam apenas o conceito de comparação e não determinam necessariamente a implementação matemática do componente.

O mecanismo interno de avaliação deverá ser definido posteriormente na especificação técnica.

9.4 Seleção não significa captura imediata

A identificação da melhor candidata não significa que a fotografia deverá ser capturada imediatamente.

Após selecionar uma candidata, o componente deverá estabelecer o **Face Lock** e passar a acompanhar exclusivamente aquela pessoa.

A candidata ainda deverá cumprir os demais requisitos necessários para atingir PRONTO.

O fluxo será conceitualmente:

DETECTANDO

↓

Várias faces detectadas

↓

Avaliação das candidatas

↓

Seleção da melhor candidata

↓

Face Lock

↓

AVALIANDO

↓

Todos os requisitos atendidos

↓

PRONTO

9.5 Não substituição durante o ciclo

Depois que uma candidata for selecionada e o Face Lock estiver ativo, o componente **não deverá substituí-la por outra face** simplesmente porque outra pessoa passou a apresentar melhores condições.

Por exemplo:

Momento 1

Pessoa A → qualidade 85

Pessoa B → qualidade 78

Candidata: Pessoa A

Face Lock: Pessoa A

Se posteriormente ocorrer:

Momento 2

Pessoa A → qualidade 74

Pessoa B → qualidade 95

o componente deverá continuar acompanhando a Pessoa A.

A Pessoa B não poderá assumir o processo de captura.

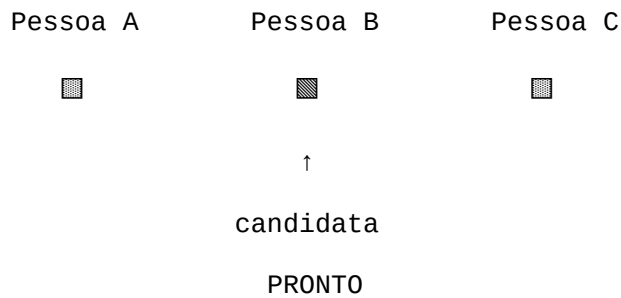
Se a Pessoa A permanecer válida, o componente continuará trabalhando com ela.

Se a Pessoa A for efetivamente perdida e o ciclo retornar a AGUARDANDO, o componente poderá iniciar uma nova seleção em que a Pessoa B eventualmente seja escolhida.

9.6 Uma única candidata em PRONTO

Mesmo que várias faces permaneçam visíveis no quiosque, o estado PRONTO deverá pertencer a **uma única candidata por vez**.

A representação visual deverá deixar isso claro:



A moldura verde deverá identificar exclusivamente a face que atende aos requisitos para captura.

As demais faces continuarão sendo apenas faces detectadas.

9.7 Entrada de novas pessoas

A entrada de uma nova pessoa no campo de visão não deverá interromper o processo em andamento.

Se uma pessoa entrar depois que uma candidata já tiver sido selecionada:

Pessoa A → Face Lock ativo

Pessoa B → entra no campo de visão

a Pessoa B será apenas uma nova face detectada.

Ela não poderá substituir a candidata atual.

Somente o retorno do componente a AGUARDANDO permitirá que uma nova seleção seja realizada.

9.8 Ausência de candidata adequada

Se existirem faces no campo de visão, mas nenhuma apresentar condições suficientes para ser selecionada, o componente deverá permanecer em processo de detecção/avaliação.

A interface deverá informar ao usuário, por meio das mensagens disponíveis, o principal motivo que impede a evolução do processo.

Por exemplo:

- "Aproxime-se."
- "Afastese um pouco."
- "Mova-se para o centro."

- "Melhore a iluminação."
- "Olhe para a câmera."
- "Aguarde o foco."

Não deverá haver seleção artificial de uma face apenas porque ela é a única disponível.

9.9 Prioridade da qualidade sobre a proximidade

A regra fundamental para o modo quiosque será:

A melhor candidata é aquela que apresenta as melhores condições para produzir uma fotografia facial adequada, e não necessariamente aquela que está mais próxima da câmera.

Essa regra deverá orientar o mecanismo de seleção e evitar que o quiosque privilegie automaticamente pessoas que estejam mais próximas do equipamento.

9.10 Regra de segurança da seleção

A seleção de uma candidata deverá ocorrer somente quando houver evidências suficientes de que ela pode ser acompanhada de maneira estável.

O componente deverá evitar selecionar uma face que esteja:

- entrando ou saindo rapidamente do enquadramento;
- parcialmente ocultada;
- excessivamente inclinada;
- muito pequena;
- muito próxima ou muito distante;
- excessivamente desfocada;
- sujeita a oscilações que dificultem seu acompanhamento.

A candidata deverá ser suficientemente identificável e estável para que o Face Lock possa assumir o acompanhamento.

9.11 Relação com os demais modos

Esta lógica de seleção múltipla será específica do modo **quiosque**.

Nos modos **autorretrato** e **assistido**, a regra será mais restritiva: para que uma face seja considerada candidata válida, deverá existir **somente uma face detectada naquele momento**, conforme definido anteriormente.

Assim:

Modo	Faces detectadas	Comportamento
Autorretrato	0	Continua procurando
Autorretrato	1	Pode avaliar a candidata

Autorretrato	>1 Não fica PRONTO
Assistido	0 Continua procurando
Assistido	1 Pode avaliar a candidata
Assistido	>1 Não fica PRONTO
Quiosque	0 Continua procurando
Quiosque	1 Avalia a face
Quiosque	>1 Seleciona a melhor candidata

Independentemente do modo, somente uma face poderá estar efetivamente vinculada ao processo de captura.

No quiosque, essa seleção será realizada automaticamente; nos demais modos, a presença de mais de uma face impedirá que o componente considere a captura PRONTO.

10. Timer e estabilidade

O `FotografoDeFACES` deverá considerar que uma pessoa real não consegue permanecer completamente imóvel durante o processo de captura. Portanto, a estabilidade exigida pelo componente deverá possuir **tolerância a pequenos movimentos naturais**, sem comprometer a qualidade da fotografia.

O objetivo não é exigir que a pessoa fique imóvel como uma estátua, mas garantir que, no momento da captura, a face esteja suficientemente estável para produzir uma fotografia adequada ao processamento posterior.

10.1 Propriedade `autoCaptureAfter`

A propriedade `autoCaptureAfter` determinará se a fotografia será capturada automaticamente e, quando aplicável, quanto tempo deverá transcorrer antes do disparo.

Se:

```
autoCaptureAfter = null
```

não haverá disparo automático.

Nesse caso, o componente poderá permanecer em PRONTO aguardando o disparo manual, realizado pelo botão de captura disponibilizado pelo próprio componente ou, opcionalmente, pelo método `capture()` disponibilizado via `useRef`.

Se:

```
autoCaptureAfter = 0
```

a fotografia deverá ser capturada automaticamente assim que o componente atingir PRONTO e confirmar que todos os requisitos necessários foram satisfeitos.

Se:

`autoCaptureAfter > 0`

o componente deverá iniciar uma contagem regressiva pelo período especificado em milissegundos.

Por exemplo:

`autoCaptureAfter = 3000`

representará uma espera de três segundos antes do disparo automático.

10.2 Início da contagem

A contagem regressiva somente poderá começar quando a candidata estiver efetivamente em condição PRONTO.

Portanto, o fluxo será:

AGUARDANDO

↓

DETECTANDO

↓

AVALIANDO

↓

PRONTO

↓

CRONOMETRANDO

↓

CAPTURANDO

A contagem **não deverá começar simplesmente porque uma face foi detectada.**

A candidata deverá primeiro atender aos critérios de qualidade e estabilidade definidos pelo componente.

10.3 Estado CRONOMETRANDO

O estado CRONOMETRANDO existirá somente quando `autoCaptureAfter` for maior que zero.

Durante esse estado, o componente deverá informar visualmente ao usuário que a captura automática está em andamento.

Exemplo:

Captura preparada.

Aguarde... 1 de 3 segundos.

Depois:

Aguarde... 2 de 3 segundos.

E finalmente:

Aguarde... 3 de 3 segundos.

A apresentação exata do texto poderá ser definida posteriormente na especificação visual, mas o componente deverá disponibilizar as informações necessárias para que tanto sua própria interface quanto a aplicação hospedeira possam apresentar o progresso da contagem.

10.4 Cronômetro no próprio componente

Quando a contagem estiver ativa, o próprio `FotografoDeFACES` deverá ser capaz de apresentar o cronômetro ao usuário.

A aplicação hospedeira não deverá ser obrigada a construir esse mecanismo.

O componente deverá apresentar, de maneira clara, que a captura está sendo preparada e quanto tempo falta para o disparo.

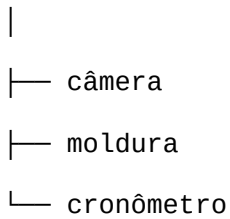
10.5 Cronômetro pelo `useRef`

As mesmas informações deverão estar disponíveis por meio da API pública do componente.

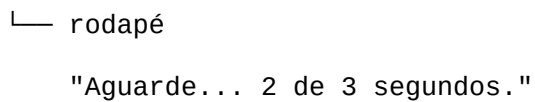
Isso permitirá que a aplicação hospedeira apresente o progresso em outra região da interface.

Por exemplo:

FotografoDeFaces



Aplicação



Dessa maneira, a aplicação poderá decidir onde deseja apresentar a informação sem depender exclusivamente da interface interna do componente.

10.6 Movimentos durante a contagem

Durante CRONOMETRANDO, o componente deverá continuar monitorando a candidata.

Pequenos movimentos naturais deverão ser tolerados.

Exemplos de movimentos que não necessariamente deverão interromper a contagem:

- pequeno deslocamento da cabeça;
- pequena alteração de posição;
- movimento natural dos olhos;
- pequena variação do enquadramento;
- pequenas oscilações da posição do rosto.

O objetivo é evitar uma experiência excessivamente rígida.

10.7 Movimento incompatível com uma boa fotografia

Se durante a contagem ocorrer um movimento suficientemente grande para comprometer a qualidade da fotografia, o componente deverá interromper a contagem.

Por exemplo:

CRONOMETRANDO

↓

movimento excessivo

↓

contagem interrompida

↓

AGUARDANDO

O componente deverá então iniciar novamente o processo de identificação de uma candidata adequada.

Isso também significa liberar o Face Lock anterior, pois a pessoa poderá ter se afastado ou deixado de ser a melhor referência para um novo ciclo.

10.8 Perda da candidata durante a contagem

Se a candidata desaparecer durante CRONOMETRANDO, o disparo deverá ser cancelado.

Uma pequena falha momentânea de detecção poderá ser tolerada, mas se o componente determinar que a candidata foi efetivamente perdida, deverá:

1. interromper o cronômetro;
2. cancelar o disparo;
3. liberar o Face Lock;
4. retornar a AGUARDANDO;
5. iniciar um novo ciclo de busca.

O retorno a AGUARDANDO é deliberado: uma nova pessoa poderá ter assumido a posição diante do quiosque, portanto não devemos presumir que a candidata anterior ainda seja o alvo correto.

10.9 Retorno à estabilidade

Caso a candidata faça um movimento incompatível com a captura, o componente não deverá simplesmente continuar a contagem anterior.

O novo ciclo deverá começar novamente pela identificação de uma candidata adequada.

Assim:

PRONTO

↓

CRONOMETRANDO

↓

movimento excessivo

↓

AGUARDANDO

↓

DETECTANDO



AVALIANDO



PRONTO



CRONOMETRANDO

Isso garante que a nova fotografia seja precedida novamente por todas as verificações necessárias.

10.10 `autoCaptureAfter = 0`

Quando `autoCaptureAfter` for igual a zero, não haverá contagem regressiva.

Assim que a candidata atingir PRONTO, o componente deverá iniciar automaticamente a captura.

O fluxo será:

AVALIANDO



PRONTO



CAPTURANDO

Mesmo nesse caso, a captura não poderá ignorar os critérios de qualidade.

O valor zero representa **tempo mínimo de espera**, e não uma autorização para capturar uma fotografia que ainda não esteja pronta.

10.11 `autoCaptureAfter = null`

Quando `autoCaptureAfter` for `null`, não haverá captura automática.

O componente deverá permanecer em PRONTO até que ocorra um disparo manual.

O disparo poderá ocorrer:

- pelo botão de captura exibido pelo próprio componente, quando aplicável;
- pelo método `capture()` disponibilizado pelo `useRef`.

Em ambos os casos, o componente deverá verificar novamente se permanece em condição válida para captura.

10.12 Estabilidade nos três modos

A tolerância a movimentos deverá existir nos três modos:

- autorretrato;
- assistido;
- quiosque.

A diferença estará principalmente na forma de seleção da candidata.

No autorretrato e no assistido, a existência de mais de uma face impedirá que uma única candidata seja considerada válida.

No quiosque, múltiplas faces poderão existir, mas somente uma será selecionada e protegida pelo Face Lock.

Em todos os casos, a candidata deverá apresentar estabilidade suficiente para produzir uma fotografia facial adequada.

10.13 Regra fundamental

O princípio geral será:

O componente deve tolerar movimentos naturais, mas não deve capturar quando o movimento comprometer a qualidade da fotografia.

Assim, estabilidade não significa imobilidade absoluta.

Significa que a face deve permanecer suficientemente estável dentro dos limites definidos pelo `FotografoDeFaces` para que a fotografia resultante seja considerada adequada.

11. Regras de captura manual

A captura manual será utilizada quando o `FotografoDeFaces` estiver configurado sem disparo automático, ou seja, quando `autoCaptureAfter` estiver definido como `null`.

Nesse modo, o componente deverá apresentar ao usuário a condição atual da captura e, quando a fotografia estiver PRONTO, disponibilizar o botão de disparo.

A captura manual poderá ocorrer de duas formas:

1. pelo botão de captura apresentado pelo próprio `FotografoDeFaces`;
2. por comando externo da aplicação hospedeira por meio do `useRef`.

Ambas as formas deverão obedecer exatamente às mesmas regras internas de captura.

11.1 Condição obrigatória para capturar

O `FotografoDeFACES` somente poderá produzir uma fotografia quando estiver no estado:

`PRONTO`

Isso significa que o componente já confirmou que existe uma face candidata identificável e estável, suficientemente enquadrada, nítida, iluminada e frontal, dentro dos limites definidos para produzir uma fotografia facial adequada ao processamento posterior.

A existência do botão ou a chamada de `capture()` não deverá substituir essa validação.

11.2 Botão de captura

Quando `autoCaptureAfter = null`, o próprio componente poderá apresentar um botão de captura.

Esse botão deverá acompanhar o estado do componente.

Enquanto a fotografia não estiver `PRONTO`, o botão deverá permanecer indisponível para disparo.

Quando a condição `PRONTO` for atingida, o botão deverá tornar-se disponível.

Conceitualmente:

`AVALIANDO`

[Capturar] ← indisponível

e:

`PRONTO`

[Capturar] ← disponível

O componente poderá utilizar mensagens e recursos visuais para informar por que o disparo ainda não está disponível.

11.3 Disparo pelo useRef

A aplicação hospedeira poderá optar por não utilizar o botão interno e criar seu próprio controle de disparo.

Nesse caso, poderá utilizar:

```
fotografoRef.current.capture();
```

O método deverá executar exatamente a mesma validação realizada pelo botão interno.

Portanto, o comando externo não terá privilégio sobre a lógica interna do componente.

11.4 capture() não significa "capturar a qualquer momento"

A chamada:

```
fotografoRef.current.capture();
```

deverá ser entendida como:

Solicitar ao FotografoDeFACES que faça a captura, caso as condições para captura estejam satisfeitas.

Ela não deverá significar:

"Ignore as regras e capture imediatamente o frame atual."

Isso é fundamental para preservar a finalidade do componente.

11.5 Chamada de capture() antes de PRONTO

Se a aplicação chamar capture() enquanto o componente não estiver em PRONTO, o componente não deverá produzir fotografia.

A chamada deverá ser simplesmente ignorada, sem lançar exceção.

A aplicação poderá consultar o estado atual por meio de getState() caso precise saber por que a captura não foi realizada.

Exemplo:

Estado atual: AVALIANDO

Aplicação → capture()

Resultado:

nenhuma fotografia produzida

nenhuma exceção lançada

estado permanece sob controle do componente

A decisão evita que uma tentativa de captura inadequada provoque erro de execução na aplicação hospedeira.

11.6 Chamada de `capture()` em PRONTO

Quando `capture()` for chamado em PRONTO, o componente deverá iniciar o processo de captura.

O fluxo será:

PRONTO

↓

CAPTURANDO

↓

tratamento da fotografia

↓

FOTOGRAFIA_PRONTA

A partir do início de CAPTURANDO, novos comandos de captura deverão ser ignorados até que o processo termine.

11.7 Bloqueio durante CAPTURANDO

Enquanto o componente estiver em CAPTURANDO, o botão interno deverá permanecer indisponível e chamadas adicionais a `capture()` deverão ser ignoradas.

Isso impede situações como:

`capture()`

`capture()`

`capture()`

`capture()`

produzindo múltiplas fotografias simultaneamente.

O `FotografoDeFACES` deverá concluir o ciclo atual antes de aceitar uma nova solicitação de captura.

11.8 Resultado da captura

Uma captura manual bem-sucedida deverá resultar em uma fotografia tratada e entrar no estado:

`FOTOGRAFIA_PRONTA`

Nesse momento, a câmera ao vivo deverá deixar de ser o elemento principal da interface e a fotografia final produzida deverá ser apresentada ao usuário.

A fotografia deverá ser aquela efetivamente preparada pelo componente para utilização posterior pela aplicação hospedeira.

11.9 Falha durante a captura

Se houver algum problema durante o processo de captura ou tratamento da fotografia, o componente não deverá produzir uma fotografia inválida como se fosse uma captura bem-sucedida.

O comportamento deverá ser tratado conforme as regras de erro definidas posteriormente neste documento.

Em qualquer situação, a aplicação deverá conseguir identificar que a fotografia não foi produzida corretamente por meio do estado e das informações disponibilizadas pelo componente.

11.10 Captura manual e `autoCaptureAfter`

A existência do método `capture()` não elimina o funcionamento automático.

Quando:

`autoCaptureAfter = null`

o disparo será manual.

Quando:

`autoCaptureAfter = 0`

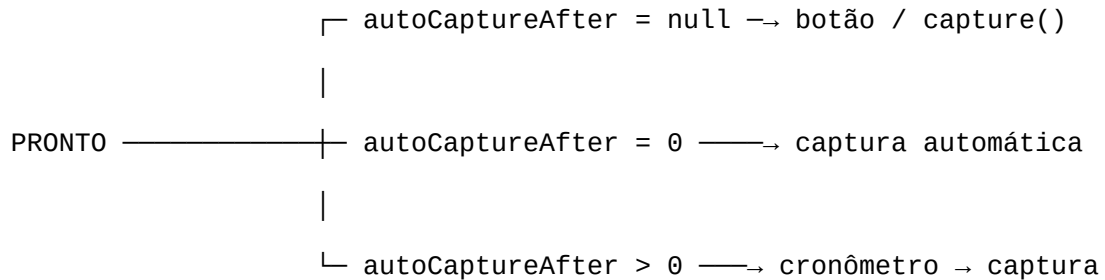
o disparo ocorrerá automaticamente assim que o estado PRONTO for alcançado.

Quando:

`autoCaptureAfter > 0`

o disparo ocorrerá automaticamente após a contagem definida.

A regra de qualidade, entretanto, permanece a mesma em todos os casos.



11.11 Regra fundamental da captura manual

O botão interno e o método `capture()` são apenas **formas diferentes de solicitar a captura**.

Nenhum deles poderá ultrapassar as regras de qualidade do componente.

A regra definitiva será:

O `FotografoDeFACES` somente captura quando considerar que a fotografia está PRONTO.

Dessa forma, a aplicação hospedeira pode controlar a experiência de uso, mas não precisa conhecer ou reproduzir internamente as regras biométricas de qualidade do componente.

12. Tratamento da fotografia

Após o disparo da fotografia, o `FotografoDeFACES` deverá realizar o tratamento necessário para transformar o frame capturado em uma fotografia facial adequada ao uso posterior pela aplicação hospedeira.

O componente será responsável por preparar a fotografia, mas **não será responsável por enviá-la ao backend**.

A aplicação hospedeira decidirá quando e como utilizar a fotografia produzida, inclusive para envio a serviços de biometria, armazenamento, atualização de cadastro ou qualquer outra finalidade.

12.1 Separação entre análise e fotografia final

Durante a utilização da câmera, o componente realizará análises contínuas para determinar se existe uma face adequada para captura.

Essas análises servem para determinar se a fotografia está PRONTO.

Depois que o disparo ocorrer, o componente deverá produzir a fotografia final utilizando o frame capturado e as informações da face que estava sendo acompanhada.

Portanto:

Câmera

↓

Deteção e avaliação contínua

↓

PRONTO

↓

Captura do frame

↓

Tratamento

↓

Fotografia final

A fotografia final deverá ser independente das imagens utilizadas apenas para análise contínua.

12.2 Seleção da região da fotografia

A fotografia final deverá ser recortada a partir da região correspondente à face candidata que foi capturada.

O recorte não deverá ser excessivamente justo.

Deverá existir uma **margem de enquadramento** ao redor da face, permitindo preservar contexto suficiente para que a imagem não fique artificialmente cortada.

A margem deverá considerar:

- dimensões da face;
- proporções do enquadramento;
- posição da face no frame;
- espaço suficiente acima da cabeça;
- espaço lateral adequado;
- pequena região abaixo da face.

O componente deverá evitar cortes que eliminem partes relevantes da cabeça ou deixem a face excessivamente próxima às bordas.

12.3 Utilização da face candidata

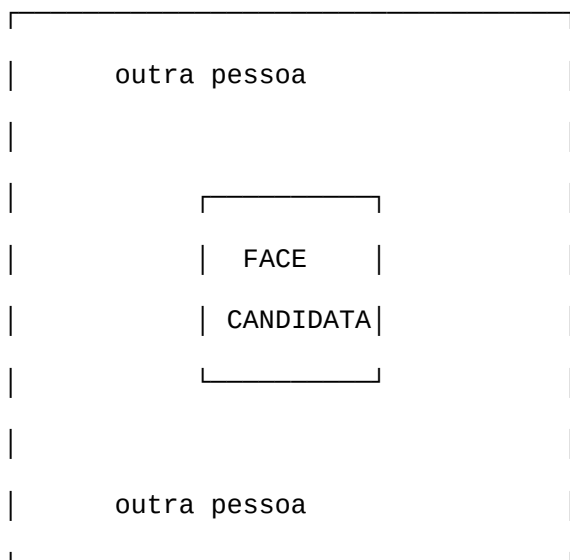
O recorte deverá utilizar como referência a face que estava efetivamente vinculada ao ciclo de captura.

No modo quiosque, isso é especialmente importante.

Se houver outras pessoas no frame no momento da captura, elas não deverão ser utilizadas para definir o recorte final.

A fotografia deverá corresponder exclusivamente à candidata protegida pelo Face Lock.

Conceitualmente:



↓

recorte da candidata

↓

fotografia final

12.4 Redimensionamento

Após o recorte, a fotografia deverá ser redimensionada para uma resolução padronizada pelo componente.

O objetivo é evitar que a aplicação hospedeira receba imagens com dimensões excessivamente variadas em função das diferentes webcams, computadores, celulares ou configurações de câmera.

O componente deverá produzir uma fotografia com dimensões previsíveis e adequadas ao processamento facial posterior.

A resolução final deverá ser definida na implementação técnica, considerando o equilíbrio entre:

- qualidade da imagem;
- desempenho;
- tamanho do arquivo;
- utilização de memória;
- compatibilidade com processamento biométrico.

12.5 Formato da fotografia

A fotografia final deverá ser disponibilizada em formato adequado para utilização web.

A implementação deverá priorizar **JPEG**, por oferecer boa relação entre qualidade e tamanho para fotografias provenientes de webcam.

O componente deverá utilizar uma qualidade de compressão suficientemente alta para evitar degradação perceptível ou prejuízo relevante ao processamento biométrico.

Como referência inicial, poderá ser utilizada qualidade na faixa de **0,85 a 0,92**, devendo o valor definitivo ser validado durante a implementação e testes.

12.6 Produção do Blob

A fotografia final deverá ser produzida como um Blob.

Esse será o objeto binário principal disponibilizado pelo componente para a aplicação hospedeira.

A aplicação poderá utilizar o Blob diretamente para montar uma requisição HTTP, FormData ou outro mecanismo de transporte.

O componente não deverá transformar obrigatoriamente a fotografia em Base64 apenas para facilitar o consumo pela aplicação.

Quando necessário, a própria aplicação poderá realizar essa conversão.

12.7 Compatibilidade com value

Embora a fotografia final seja produzida internamente como um Blob, o componente deverá manter compatibilidade com o contrato público definido para value.

O value representará sempre a **fotografia vigente e válida do componente**.

Assim, a implementação deverá compreender claramente:

Fotografia produzida

↓

value

12.8 Fotografia inicial

O componente também poderá ser iniciado com uma fotografia previamente fornecida pela aplicação.

Nesse cenário, o value inicial representará a fotografia atualmente confirmada.

O FotografoDeFACES deverá apresentar essa fotografia da mesma forma que apresentaria uma fotografia confirmada após uma captura.

Isso permite utilizar o componente não apenas para captura inicial, mas também para:

- edição de perfil;
- atualização de fotografia;
- consulta de registros;
- histórico de fotografias;
- substituição de uma fotografia anteriormente registrada.

12.9 Limpeza da fotografia

Quando o fluxo de substituição estiver habilitado e o usuário executar a ação **Limpar**, o componente deverá produzir uma situação em que não exista fotografia atualmente confirmada.

Nesse caso:

```
value = null
```

A fotografia anteriormente confirmada deverá deixar de ser considerada a fotografia válida do componente.

A interface deverá então seguir o fluxo de confirmação/cancelamento definido para a operação em andamento.

12.10 Não envio ao backend

O `FotografoDeFACES` **não deverá possuir responsabilidade sobre comunicação com o backend da aplicação.**

Não deverá existir dentro do componente uma regra específica para:

- endpoint;
- autenticação;
- token;
- API de biometria;
- cadastro de pessoa;
- reconhecimento facial;
- atualização de usuário;
- persistência de imagem.

O componente deverá apenas produzir e disponibilizar a fotografia.

A aplicação hospedeira será responsável por decidir o que fazer com ela.

Exemplo:

`FotografoDeFACES`

```
|  
| fotografia final  
↓
```

Aplicação hospedeira

```
|  
|— reconhecimento facial  
|— atualização de perfil  
|— armazenamento  
|— histórico  
|— qualquer outra finalidade
```

12.11 Regra fundamental

O tratamento da fotografia deverá obedecer ao seguinte princípio:

O FotoGrafoDeFaces prepara a fotografia; a aplicação hospedeira decide o que fazer com ela.

A fotografia produzida deverá estar recortada, tratada, redimensionada e comprimida de maneira consistente, ficando pronta para ser utilizada pela aplicação sem que esta precise conhecer ou reproduzir a lógica interna de preparação da imagem.

13. Fotografia final

A fotografia final é o resultado produzido pelo FotoGrafoDeFaces após uma captura considerada válida e seu respectivo tratamento.

A fotografia produzida deverá estar recortada, tratada, redimensionada e comprimida conforme as regras definidas pelo componente, ficando pronta para utilização pela aplicação hospedeira.

O componente deverá apresentar a fotografia produzida ao usuário e disponibilizá-la por meio de sua interface pública.

13.1 Fotografia produzida

Quando o componente realizar uma captura com sucesso, deverá produzir uma fotografia tratada conforme as regras estabelecidas na Parte 12.

Essa fotografia deverá ser apresentada visualmente ao usuário.

O componente deverá deixar de exibir a câmera ao vivo como conteúdo principal e apresentar a fotografia produzida.

A fotografia produzida passará a representar o valor atual do componente quando esse valor for atribuído a `value`.

A alteração deverá ser comunicada à aplicação por meio de `onChange`, respeitando o padrão de componente controlado.

13.2 `value`

A propriedade `value` representa sempre o valor atual do componente, independentemente do estado em que ele se encontre.

O `value` poderá ser:

- `null`, quando não houver fotografia atualmente atribuída ao componente;
- um `Blob` contendo uma fotografia válida.

O `value` poderá ser fornecido inicialmente pela aplicação, inclusive quando o componente for carregado já contendo uma fotografia.

Assim, são válidas as seguintes situações:

```
value = null
```

ou:

```
value = Blob
```

A existência de `value` não deverá ser interpretada como indicação de que a fotografia foi necessariamente produzida pelo próprio componente naquele momento.

Uma fotografia poderá estar em `value` porque:

- foi fornecida inicialmente pela aplicação;
- foi produzida pelo componente;
- foi confirmada após uma operação de alteração;
- foi restaurada após uma operação de cancelamento;
- foi atualizada pela aplicação como parte do padrão controlado do componente.

13.3 Relação entre `value` e `onChange`

O `FotografoDeFaces` deverá comportar-se como um componente controlado.

Conceitualmente:

```
value={foto}
```

```
onChange={setFoto}
```

A aplicação hospedeira será responsável por manter o valor externo.

Quando o componente precisar alterar o valor atual, deverá comunicar a alteração por meio de `onChange`.

Por exemplo, quando uma nova fotografia for produzida:

```
Fotografia produzida
```

↓

```
onChange(novaFotografia)
```

↓

```
Aplicação atualiza seu estado
```

↓

```
value = novaFotografia
```

Da mesma forma, quando o usuário executar uma ação que resulte em remoção da fotografia:

Limpar

↓

`onChange(null)`

↓

Aplicação atualiza seu estado

↓

`value = null`

O componente não deverá considerar concluída uma alteração externa apenas por ter emitido `onChange`. A referência pública do valor continuará sendo o `value` recebido pela aplicação.

13.4 Fotografia fornecida inicialmente

Quando a aplicação iniciar o componente com:

`value = null`

o componente deverá iniciar sem fotografia e seguir o fluxo normal de captura.

Quando a aplicação iniciar o componente com:

`value = Blob`

o componente deverá reconhecer esse valor como uma fotografia válida atualmente disponível.

Nesse caso, deverá apresentar a fotografia como conteúdo principal, da mesma maneira que apresentaria uma fotografia produzida após uma captura.

O componente não deverá exigir uma nova captura somente porque a fotografia foi fornecida pela aplicação.

13.5 Fotografia em operação de revisão

A expressão "fotografia em avaliação" não deverá representar uma terceira fotografia ou um valor independente de `value`.

Ela deverá representar apenas uma situação operacional em que o componente está permitindo que o usuário decida se deseja manter ou abandonar uma alteração.

Durante uma operação de revisão, `value` continuará representando sempre o valor atual do componente.

Por isso, é perfeitamente válida a situação:

```
value = Blob
```

```
value_rollback = Blob
```

Por exemplo:

Foto A = fotografia atualmente exibida

↓

Trocar

↓

```
value = null
```

```
value_rollback = Foto A
```

↓

nova captura

↓

```
value = Foto B
```

```
value_rollback = Foto A
```

↓

[Confirmar] [Cancelar]

Nesse momento:

- `value` representa a nova fotografia atualmente exibida;
- `value_rollback` representa a fotografia anterior que poderá ser restaurada;
- a operação ainda está sujeita à decisão de Confirmar ou Cancelar.

Portanto, não existe uma fotografia separada e escondida denominada "fotografia em avaliação".

Existe apenas o `value` atual e, quando necessário, uma referência interna `value_rollback` para permitir o cancelamento da operação.

13.6 Confirmar

Quando o usuário selecionar Confirmar, o valor atualmente representado por `value` deverá permanecer como valor válido do componente.

A referência interna `value_rollback` deverá ser descartada:

```
value_rollback = null
```

Por exemplo:

Antes:

```
value = Foto B
```

```
value_rollback = Foto A
```

↓ Confirmar

Depois:

```
value = Foto B
```

```
value_rollback = null
```

A partir desse momento, a Foto B passa a ser simplesmente o valor atual do componente.

Não deverá existir qualquer distinção conceitual entre uma fotografia que acabou de ser confirmada e uma fotografia que já estava no componente há algum tempo.

13.7 Cancelar

Quando o usuário selecionar Cancelar, o componente deverá solicitar à aplicação, por meio de `onChange`, a restauração do valor armazenado em `value_rollback`.

Por exemplo:

```
value = Foto B
```

```
value_rollback = Foto A
```

↓ Cancelar

`onChange(Foto A)`

Após a aplicação atualizar o `value`, o componente deverá limpar a referência de `rollback`:

`value = Foto A`

`value_rollback = null`

Quando não houver fotografia anterior:

`value = Foto B`

`value_rollback = null`

o cancelamento deverá solicitar:

`onChange(null)`

resultando em:

`value = null`

`value_rollback = null`

13.8 Limpar

Quando o usuário selecionar `Limpar`, o componente deverá solicitar:

`onChange(null)`

O `value` passará a representar `null` após a atualização realizada pela aplicação.

Quando essa ação fizer parte de uma operação de revisão, o componente deverá manter temporariamente o valor anterior em `value_rollback`, permitindo que o usuário posteriormente escolha entre `Confirmar` ou `Cancelar`.

Exemplo:

Antes:


```
value = Foto A
value_rollback = null
```

↓ Limpar

```
value = null
value_rollback = Foto A
```

Nesse cenário, o componente poderá apresentar:

```
[ Confirmar ] [ Cancelar ]
```

Se o usuário confirmar, o valor atual null será mantido e:

```
value_rollback = null
```

Se o usuário cancelar, deverá ser solicitada a restauração:

```
onChange(Foto A)
```

resultando em:

```
value = Foto A
value_rollback = null
```

13.9 Trocar

Quando o usuário selecionar Trocar, o componente deverá preservar temporariamente o value atual em value_rollback e iniciar um novo ciclo de captura.

Exemplo:

```
value = Foto A
value_rollback = null
```

↓ Trocar

```
value = null  
value_rollback = Foto A
```

Após uma nova captura:

```
value = Foto B  
value_rollback = Foto A
```

O componente deverá então disponibilizar:

[Confirmar] [Cancelar]

Se o usuário confirmar:

```
value = Foto B  
value_rollback = null
```

Se cancelar:

```
onChange(Foto A)
```

e, após a atualização:

```
value = Foto A  
value_rollback = null
```

13.10 Regra fundamental

O `FotografoDeFACES` deverá manter uma única referência pública para o valor atual: `value`.

Não deverá existir uma propriedade pública separada para "fotografia confirmada", "fotografia em avaliação" ou "fotografia anterior".

A distinção necessária durante uma operação de revisão será exclusivamente interna:

value

representa o valor atual do componente.

value_rollback

representa temporariamente o valor anterior que poderá ser restaurado por Cancelar.

Assim, todas as combinações abaixo são legítimas:

value	value_rollback	Significado
null	null	Componente sem fotografia e sem alteração em andamento
Blob	null	Componente contendo uma fotografia sem alteração em andamento
null	Blob	Alteração em andamento, sem nova fotografia atualmente atribuída
Blob	Blob	Alteração em andamento, com nova fotografia atualmente atribuída

A combinação Blob + Blob é particularmente importante no fluxo de substituição de uma fotografia existente:

Foto A

↓

Trocar

↓

value = null

value_rollback = Foto A

↓

nova captura

↓

value = Foto B

value_rollback = Foto A

↓

Confirmar / Cancelar

Portanto, `value_rollback` não representa o valor válido do componente. Ele representa somente uma referência temporária utilizada para permitir o cancelamento de uma alteração.

13.11 Regra de consistência

A máquina de estados deverá se ajustar ao valor atual de `value`, e não redefinir arbitrariamente o significado desse valor.

Sempre que o `value` recebido pelo componente mudar:

- `null` deverá representar ausência de fotografia;
- um `Blob` válido deverá representar presença de fotografia.

A partir desse valor, o componente deverá manter sua apresentação e seu fluxo coerentes com a situação atual.

O estado interno não deverá transformar `value` em um conceito diferente daquele que a aplicação efetivamente forneceu.

O `value` será sempre a fonte pública de verdade sobre a fotografia atualmente representada pelo componente.

14. Mensagens exibidas

O `FotografoDeFACES` deverá estabelecer um diálogo claro e objetivo com o usuário durante todo o seu funcionamento.

As mensagens deverão informar a situação atual do componente, orientar o posicionamento da pessoa diante da câmera, informar situações de espera e comunicar o resultado das operações realizadas.

As mensagens são uma responsabilidade do próprio `FotografoDeFACES`, pois o componente conhece o contexto interno necessário para determinar o que deve ser informado ao usuário.

A aplicação hospedeira, entretanto, poderá optar por apresentar essas informações em outros locais da interface.

14.1 Exibição das mensagens

A propriedade:

`showMessages`

determinará se as mensagens serão apresentadas visualmente pelo próprio componente.

Quando:

```
showMessages = true
```

o `FotografoDeFaces` deverá apresentar suas mensagens em sua própria interface.

Quando:

```
showMessages = false
```

o componente não deverá apresentar visualmente suas mensagens, mas continuará produzindo e disponibilizando as informações para a aplicação por meio da API pública do componente.

Assim, ocultar as mensagens não significa desativar a lógica de comunicação do componente.

14.2 Mensagem atual

O componente deverá possuir internamente uma mensagem atual, correspondente à situação mais relevante naquele momento.

Essa mensagem poderá mudar continuamente durante o ciclo de operação.

Exemplos:

Aguardando pessoa...

Rosto detectado.

Mova-se um pouco para a direita.

Aproxime-se um pouco.

Melhore a iluminação.

Aguarde o foco.

Captura OK.

Aguarde... 2 de 3 segundos.

Fotografia capturada.

Fotografia pronta.

Fotografia confirmada.

Fotografia cancelada.

Fotografia limpa.

As mensagens efetivamente utilizadas deverão ser definidas de acordo com os estados e regras estabelecidos nas demais partes deste documento.

14.3 Uma orientação principal por vez

Quando houver múltiplos problemas simultaneamente, o componente deverá priorizar a orientação mais importante para que o usuário consiga alcançar uma fotografia válida.

Por exemplo, se o rosto estiver:

- descentralizado;
- pequeno;
- escuro;
- e com pose inadequada;

o componente não deverá apresentar quatro instruções simultâneas.

Deverá selecionar a orientação mais relevante naquele momento, conduzindo o usuário progressivamente até a condição PRONTO.

Isso reduz a poluição visual e torna o diálogo mais compreensível.

14.4 Mensagens relacionadas ao posicionamento

O componente poderá orientar o usuário sobre:

- ausência de face;
- excesso de faces;
- deslocamento para esquerda;
- deslocamento para direita;
- deslocamento para cima;
- deslocamento para baixo;
- aproximação;
- afastamento;
- iluminação inadequada;
- pose inadequada;
- movimento excessivo;
- necessidade de aguardar estabilização;
- enquadramento inadequado.

As mensagens deverão ser objetivas e adequadas ao contexto de operação.

14.5 Mensagens relacionadas ao estado

Além das orientações de posicionamento, o componente deverá informar mudanças importantes em seu ciclo de vida.

Exemplos:

AGUARDANDO

"Aguardando pessoa..."

DETECTANDO

"Rosto detectado."

AVALIANDO

"Ajuste sua posição."

CRONOMETRANDO

"Aguarde... 2 de 3 segundos."

PRONTO

"Captura OK."

CAPTURANDO

"Capturando..."

FOTOGRAFIA_PRONTA

"Fotografia pronta."

ERRO

"Não foi possível concluir a captura."

As mensagens definitivas poderão ser refinadas durante a implementação, sem alterar a responsabilidade do componente.

14.6 Cronômetro

Quando `autoCaptureAfter` estiver configurado com valor superior a zero e o componente estiver no estado `CRONOMETRANDO`, a mensagem deverá informar o progresso do período de espera.

Por exemplo, para três segundos:

Aguarde... 1 de 3 segundos.

Aguarde... 2 de 3 segundos.

Aguarde... 3 de 3 segundos.

O próprio componente deverá poder apresentar visualmente esse progresso.

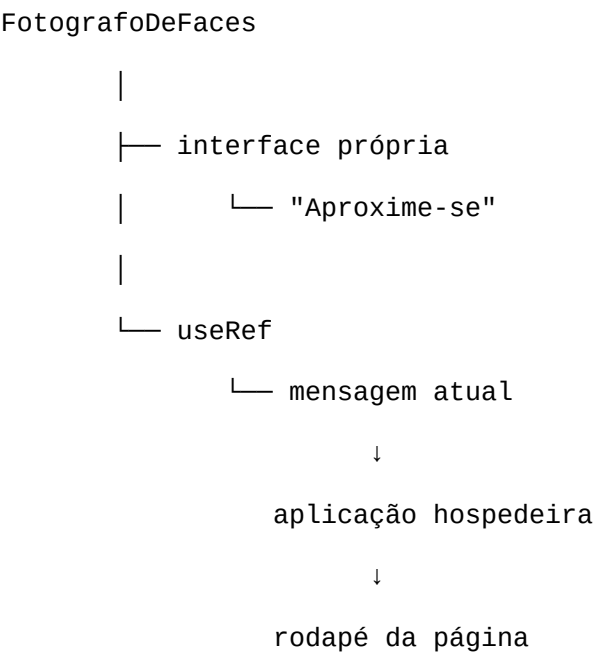
A aplicação também deverá poder obter as informações correspondentes por meio da API pública, permitindo que um elemento externo apresente seu próprio cronômetro, caso deseje.

14.7 Mensagens e useRef

As informações de comunicação do componente deverão estar disponíveis por meio do `useRef`.

A aplicação poderá consultar o estado atual e as informações relacionadas à mensagem por meio do método de consulta definido para a API pública.

Isso permitirá situações como:

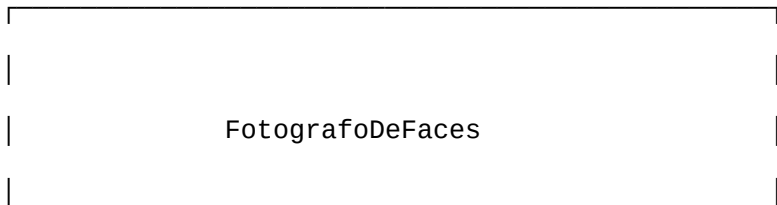


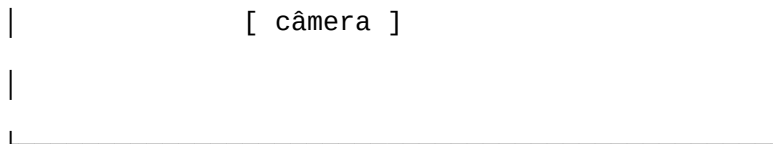
Dessa forma, a aplicação não precisará reproduzir a lógica que determina qual mensagem deve ser exibida.

14.8 Mensagens externas

A aplicação hospedeira poderá utilizar as informações disponibilizadas pelo componente para apresentar a mensagem em outro local da interface.

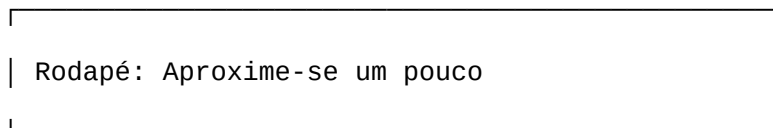
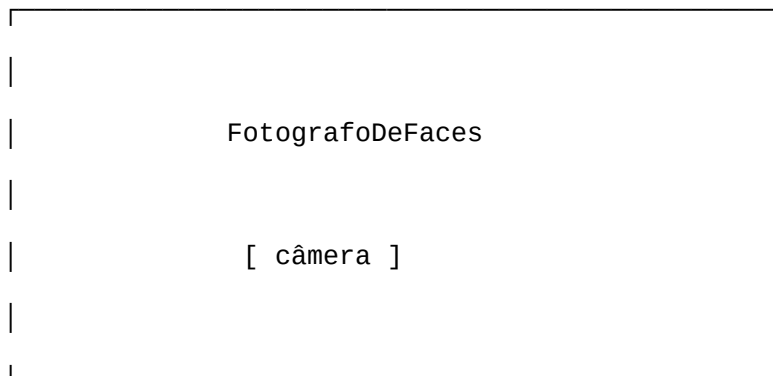
Por exemplo:





Aproxime-se um pouco

Ou:



A segunda possibilidade será especialmente útil quando a aplicação desejar construir uma interface própria ao redor do componente.

14.9 Mensagens não substituem os indicadores visuais

As mensagens textuais deverão complementar os indicadores visuais do `FotografoDeFaces`.

O componente poderá utilizar simultaneamente:

- mensagem textual;
- moldura da face;
- cor da moldura;
- guia de enquadramento;
- cronômetro;
- animações;
- fotografia revelada;

- botões de ação.

No modo quiosque, por exemplo, uma pessoa poderá compreender que foi selecionada pelo componente principalmente pela **moldura verde** sobre seu rosto, enquanto a mensagem textual reforça a situação.

14.10 Mensagens e múltiplas faces no quiosque

No modo quiosque, o componente poderá detectar diversas faces simultaneamente.

As mensagens não deverão induzir os demais indivíduos presentes a acreditar que foram selecionados.

A comunicação visual deverá estar associada exclusivamente à face candidata selecionada pelo `FotografoDeFaces`.

A moldura amarela indicará uma face detectada ainda não considerada PRONTO.

Quando uma face for selecionada como candidata válida e alcançar PRONTO, sua moldura passará a verde.

As demais faces não deverão receber indicação visual de que são o alvo da captura.

14.11 Resultado da captura

Após uma captura bem-sucedida, o componente deverá comunicar claramente o resultado.

Exemplos:

Fotografia capturada.

ou:

Fotografia pronta.

A fotografia deverá então ser apresentada na interface de revelação, conforme definido na Parte 13.

A moldura da fotografia revelada poderá utilizar a cor azul, conforme definido anteriormente, estabelecendo uma distinção visual entre:

- amarelo → face detectada;
- verde → face PRONTO;
- azul → fotografia capturada com sucesso;
- vermelho → falha na captura.

14.12 Mensagens de erro

Quando uma captura não puder ser concluída, a mensagem deverá informar o usuário de maneira compreensível, evitando expor detalhes técnicos desnecessários.

Exemplo:

Não foi possível concluir a captura. Tente novamente.

Informações técnicas eventualmente necessárias para diagnóstico deverão estar disponíveis à aplicação por meio dos mecanismos apropriados, sem transformar a mensagem apresentada ao usuário em uma mensagem técnica.

14.13 Princípio geral

O sistema de mensagens deverá obedecer ao seguinte princípio:

O `FotografoDeFaces` deve saber o que está acontecendo, saber o que o usuário precisa fazer e disponibilizar essa informação de maneira clara, tanto em sua própria interface quanto para a aplicação hospedeira.

A propriedade `showMessages` controlará somente a apresentação das mensagens pelo próprio componente.

A aplicação continuará podendo consultar as informações por meio da API pública, independentemente de as mensagens estarem visíveis ou ocultas na interface do `FotografoDeFaces`.

15. Eventos / callbacks

O `FotografoDeFaces` deverá utilizar eventos e callbacks para comunicar à aplicação hospedeira alterações relevantes ocorridas durante sua execução.

O principal evento público será `onChange`, responsável por comunicar alterações no `value` do componente.

15.1 `onChange`

O evento `onChange` deverá ser utilizado sempre que o componente precisar comunicar à aplicação uma alteração no valor atual de `value`.

O `onChange` não deverá representar exclusivamente uma confirmação de fotografia.

Ele deverá representar a alteração do valor atual do componente, seguindo o padrão de componentes controlados do React.

O valor recebido pelo callback deverá corresponder ao novo valor que o componente deseja que a aplicação mantenha.

Exemplos:

`null` → Foto A

quando uma nova fotografia for produzida;

Foto A → `null`

quando o usuário executar Limpar;

Foto A → Foto B

quando uma nova fotografia substituir a fotografia atualmente representada;

Foto B → Foto A

quando uma operação de alteração for cancelada e a fotografia anterior precisar ser restaurada.

15.2 Fluxo controlado

O `FotografoDeFaces` deverá seguir o padrão de componente controlado do React.

A aplicação hospedeira será responsável por manter o valor:

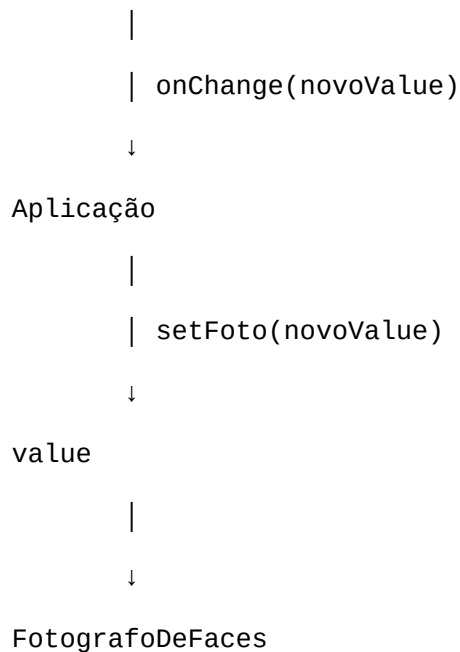
```
const [foto, setFoto] = useState(null);
```

e fornecerá o valor e o callback ao componente:

```
<FotografoDeFaces  
  value={foto}  
  onChange={setFoto}  
>
```

O fluxo será:

`FotografoDeFaces`



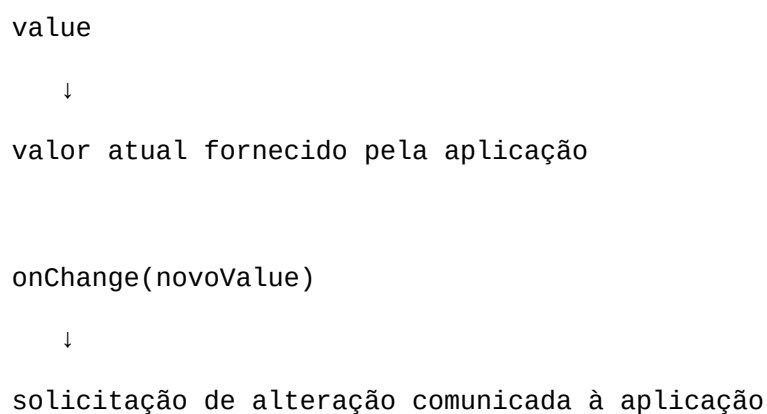
Assim:

1. o componente identifica que seu valor atual precisa ser alterado;
2. o componente dispara `onChange(novoValue)`;
3. a aplicação recebe o novo valor;
4. a aplicação atualiza seu próprio estado;
5. o novo valor é novamente fornecido ao componente por meio de `value`;
6. o componente passa a operar com esse novo `value`.

O componente não deverá considerar o simples disparo de `onChange` como substituição imediata de sua fonte externa de verdade.

15.3 value e onChange

`value` e `onChange` deverão ser tratados como partes complementares do contrato controlado do componente:



Portanto, o componente deverá sempre trabalhar com o `value` atualmente recebido.

Quando precisar alterar esse valor, deverá comunicar a intenção por `onChange`.

A aplicação poderá aceitar, rejeitar, transformar ou postergar essa alteração conforme sua própria lógica, embora o comportamento esperado para o uso normal seja atualizar seu estado e devolver o novo valor ao componente.

15.4 `onChange` durante Trocar, Limpar, Confirmar e Cancelar

As ações de revisão deverão respeitar o mesmo contrato.

Trocar

Ao iniciar uma troca, o componente poderá solicitar:

```
onChange(null)
```

fazendo com que o `value` deixe de conter a fotografia anterior e permitindo o início de um novo ciclo de captura.

Após a produção da nova fotografia:

```
onChange(novaFoto)
```

Nesse momento poderá existir:

```
value = novaFoto
```

```
value_rollback = fotoAnterior
```

Limpar

Ao selecionar Limpar:

```
onChange(null)
```

O novo `value` será `null` após a aplicação atualizar seu estado.

Caso exista `value_rollback`, ele permanecerá internamente disponível para eventual Cancelar.

Confirmar

Confirmar não implica necessariamente um novo `onChange`.

Se o `value` atual já representar a fotografia que deverá permanecer no componente, a confirmação poderá apenas encerrar a operação de revisão e limpar `value_rollback`.

Por exemplo:

```
value = Foto B
```

```
value_rollback = Foto A
```

↓ Confirmar

```
value = Foto B
```

```
value_rollback = null
```

Nenhuma alteração de `value` ocorreu nesse momento.

Cancelar

Cancelar deverá provocar `onChange` somente quando for necessário restaurar um valor diferente do `value` atual.

Por exemplo:

```
value = Foto B
```

```
value_rollback = Foto A
```

↓ Cancelar

```
onChange(Foto A)
```

Após a aplicação devolver `value = Foto A`, o componente deverá limpar `value_rollback`.

15.5 Regra fundamental

O `onChange` deverá comunicar **mudanças no valor atual do componente**, e não mudanças em uma categoria denominada "fotografia confirmada".

O contrato fundamental será:

```
value = valor atual
```

onChange = comunicação de uma nova proposta de valor

Dessa forma, o `FotografoDeFACES` permanecerá alinhado ao comportamento esperado de componentes controlados do React e poderá ser utilizado tanto para captura de uma nova fotografia quanto para edição, substituição, limpeza e restauração de fotografias existentes.

16. Comandos pelo `useRef`

O `FotografoDeFACES` deverá disponibilizar uma API imperativa por meio de `useRef`, permitindo que a aplicação hospedeira consulte informações e solicite determinadas ações.

Essa API deverá complementar as propriedades (props) e os eventos do componente, sem substituir o fluxo declarativo normalmente utilizado pelo React.

A aplicação poderá, por exemplo, utilizar botões, caixas de seleção ou outros componentes de interface para comandar ou consultar o `FotografoDeFACES`.

16.1 Referência do componente

A aplicação poderá obter uma referência para o `FotografoDeFACES` por meio de `useRef`.

Conceitualmente:

```
const fotografoRef = useRef(null);
```

```
<FotografoDeFACES ref={fotografoRef} />
```

A API exposta pela referência deverá ser pequena, previsível e documentada.

16.2 `capture()`

Solicita ao `FotografoDeFACES` que realize a captura da fotografia.

O comando não deverá ignorar as regras de qualidade do componente.

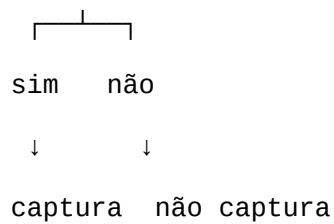
A captura somente poderá ocorrer quando o componente estiver no estado `PRONTO`.

Portanto:

```
capture()
```

↓

PRONTO?



Quando o componente não estiver PRONTO, `capture()` deverá simplesmente não realizar a captura.

Não deverá lançar exceção apenas porque a fotografia ainda não está pronta.

A aplicação poderá consultar `getState()` para compreender o estado atual e identificar por que a captura não pode ser realizada.

Captura automática

Quando `autoCaptureAfter` estiver configurado, o próprio componente será responsável pelo disparo automático.

Nesse cenário, a aplicação não precisa chamar `capture()`.

Captura manual

Quando não houver captura automática, o próprio `FotografoDeFACES` poderá apresentar seu botão de captura.

Se a aplicação desejar controlar o disparo por meio de outro elemento da interface, poderá chamar:

```
fotografoRef.current.capture();
```

Assim, haverá duas possibilidades de captura manual:

Botão interno do `FotografoDeFACES`

↓
`capture()`

ou:

Botão externo da aplicação

↓
`fotografoRef.current.capture()`

Em ambos os casos, a mesma regra permanece: somente uma fotografia cuja candidata esteja PRONTO poderá ser capturada.

16.3 restart()

Solicita que o `FotografoDeFACES` abandone o ciclo atual e reinicie seu ciclo de operação.

O comportamento deverá retornar o componente ao estado:

AGUARDANDO

A partir daí, uma nova busca por uma face candidata deverá começar.

Isso é particularmente importante no modo quiosque.

Após uma fotografia ser produzida e processada pela aplicação hospedeira, esta poderá determinar que o componente está liberado para receber outra pessoa:

```
fotografoRef.current.restart();
```

No quiosque, isso permitirá o ciclo:

AGUARDANDO

↓

pessoa A

↓

captura

↓

FOTOGRAFIA_PRONTA

↓

aplicação processa

↓

restart()

↓

AGUARDANDO

↓

pessoa B

O `restart()` também poderá ser utilizado quando a aplicação determinar que o ciclo atual deve ser abandonado e um novo ciclo precisa começar.

16.4 `setFullscreen(ativo)`

Controla a apresentação do componente em tela cheia.

Exemplo:

```
fotografoRef.current.setFullscreen(true);
```

e:

```
fotografoRef.current.setFullscreen(false);
```

O uso de um único método com parâmetro booleano deverá ser mantido, evitando métodos separados para entrar e sair da tela cheia.

Esse recurso será especialmente importante no modo quiosque, no qual a interface poderá ocupar toda a área disponível.

A disponibilidade real da tela cheia deverá respeitar as regras e limitações impostas pelo navegador.

16.5 `getState()`

Retorna um retrato atual do estado do `FotografoDeFaces`.

O resultado deverá conter informações suficientes para que a aplicação consiga compreender o que está acontecendo sem precisar consultar diversos métodos individuais.

Conceitualmente:

```
const state = fotografoRef.current.getState();
```

O objeto retornado poderá conter informações como:

```
{  
  state,  
  message,
```

```
    value,  
    quality,  
    timer,  
    candidate,  
    mode  
}
```

O contrato definitivo desse retorno já estabelecido para o componente deverá ser mantido, podendo receber informações adicionais futuramente caso exista necessidade real.

O retorno deverá representar o estado atual do componente naquele momento.

16.6 `getValue()`

Retorna o valor atual do componente.

Exemplo:

```
const value = fotoGrafoRef.current.getValue();
```

O método deverá retornar exatamente o `value` atual:

- um `Blob`, quando houver uma fotografia atualmente atribuída;
- `null`, quando não houver fotografia atualmente atribuída.

`getValue()` deverá representar o mesmo valor público fornecido pela propriedade `value`.

Por exemplo:

```
value = Foto A
```

```
getValue()
```

↓

```
Foto A
```

ou:

```
value = null
```

`getValue()`

↓

`null`

O método não deverá retornar exclusivamente uma fotografia "confirmada", nem deverá estabelecer qualquer conceito diferente daquele representado por `value`.

A aplicação poderá utilizar `getValue()` quando precisar consultar explicitamente o valor atual sem depender de uma alteração de `onChange`.

16.7 `getMessage()`

Retorna a mensagem atual produzida pelo componente.

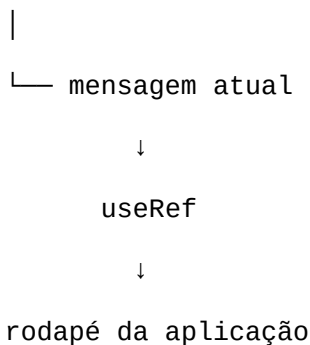
Exemplo:

```
const message = fotoGrafoRef.current.getMessage();
```

Esse recurso permite que a aplicação apresente a mensagem em outro local da interface.

Por exemplo:

FotografoDeFaces



O método deverá retornar a mensagem correspondente ao momento atual do componente.

Sua disponibilidade deverá complementar a propriedade `showMessages`: a aplicação poderá consultar a mensagem por `useRef` mesmo que opte por não exibi-la dentro da própria interface do `FotografoDeFaces`.

16.8 getQuality()

Permite consultar as informações de qualidade utilizadas pelo componente para avaliar a fotografia.

A consulta poderá disponibilizar informações relacionadas a:

- presença de face;
- quantidade de faces;
- enquadramento;
- tamanho da face;
- nitidez;
- iluminação;
- pose;
- estabilidade;
- condição geral de prontidão.

O resultado deverá ser informativo e não deverá permitir que a aplicação altere diretamente os critérios internos de qualidade.

A decisão sobre estar PRONTO continuará pertencendo exclusivamente ao `FotografoDeFaces`.

16.9 getTimer()

Quando houver cronômetro ativo, a aplicação poderá consultar seu estado atual.

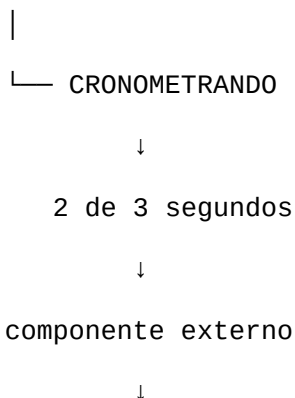
Exemplo:

```
const timer = fotografoRef.current.getTimer();
```

Esse recurso deverá permitir que a aplicação construa uma apresentação externa, caso deseje.

Por exemplo:

`FotografoDeFaces`



"Aguarde 2 de 3 segundos"

O próprio componente continuará podendo apresentar seu cronômetro internamente.

Quando não houver cronômetro ativo, o retorno deverá representar essa condição de acordo com o contrato definido para `getTimer()`.

16.10 Controle de propriedades em tempo de execução

As propriedades que forem definidas como reativas deverão continuar sendo controladas pelo fluxo normal do React.

Por exemplo:

```
<FotografoDeFaces
  mode={mode}
  showMessages={showMessages}
  showFramingGuide={showFramingGuide}
/>
```

Um combobox ou checkbox externo poderá alterar os valores:

controle externo

↓

estado React

↓

prop

↓

FotografoDeFaces

Não será necessário criar métodos no `useRef` para modificar propriedades que já possam ser controladas adequadamente por props.

Isso mantém a API imperativa menor e alinhada ao padrão React.

16.11 `useRef` não poderá burlar a máquina de estados

Nenhum comando público deverá permitir que a aplicação force artificialmente uma transição inválida.

Por exemplo, não deverá existir:

```
fotografoRef.current.forceReady();
```

nem:

```
fotografoRef.current.forceCapture();
```

O componente continuará responsável por suas próprias regras.

A aplicação poderá solicitar:

```
capture()
```

mas o componente decidirá:

Estou PRONTO?

↓

Sim → capturar

Não → ignorar

Esse princípio é fundamental para preservar a confiabilidade da fotografia produzida.

16.12 API inicial do useRef

Com as decisões atuais, a API pública inicial será composta pelos seguintes comandos e consultas:

Método	Finalidade
<code>capture()</code>	Solicitar captura quando o componente estiver PRONTO
<code>restart()</code>	Retornar o componente ao ciclo de AGUARDANDO
<code>setFullscreen(boolean)</code>	Ativar ou desativar apresentação em tela cheia
<code>getState()</code>	Consultar o estado completo atual
<code>getValue()</code>	Consultar o <code>value</code> atual
<code>getMessage()</code>	Consultar a mensagem atual
<code>getQuality()</code>	Consultar os indicadores de qualidade
<code>getTimer()</code>	Consultar o estado do cronômetro

Essa API deverá permanecer pequena e objetiva.

16.13 Princípio geral

A API imperativa deverá obedecer a uma regra:

O `useRef` permite à aplicação consultar o `FotografoDeFACES` e solicitar ações, mas não permite que a aplicação assuma o controle de suas regras internas.

O componente continuará sendo responsável por:

- detectar faces;
- selecionar candidatos;
- avaliar qualidade;
- controlar o Face Lock;
- controlar estabilidade;
- determinar PRONTO;
- controlar o cronômetro;
- realizar a captura;
- tratar a fotografia;
- controlar `value_rollback` quando houver uma operação de alteração;
- apresentar as informações visuais;
- manter sua máquina de estados consistente.

17. Contrato da fotografia produzida

O `FotografoDeFACES` será responsável por produzir uma fotografia facial tratada, padronizada e adequada para utilização posterior pela aplicação hospedeira.

O componente **não será responsável por enviar a fotografia ao backend**, realizar reconhecimento de identidade, gerar embeddings ou executar qualquer processamento biométrico.

Sua responsabilidade termina quando disponibiliza uma fotografia final que atenda aos critérios estabelecidos neste documento.

A aplicação hospedeira será responsável por decidir o que fazer com essa fotografia.

17.1 Responsabilidade do `FotografoDeFACES`

Ao concluir uma captura válida, o componente deverá:

1. selecionar o frame considerado adequado;
2. identificar a face que deverá compor a fotografia;
3. realizar o recorte;
4. aplicar a margem definida pelo componente;
5. redimensionar a imagem para o padrão de saída;
6. aplicar o tratamento mínimo necessário;
7. gerar a fotografia final;

8. disponibilizar essa fotografia para a aplicação.

O componente não deverá:

- enviar a fotografia ao backend;
- conhecer a URL do serviço biométrico;
- realizar autenticação da requisição;
- interpretar o resultado do reconhecimento facial;
- decidir se a pessoa está autorizada ou não;
- determinar o que a aplicação deverá fazer com a fotografia.

17.2 Formato da fotografia

A fotografia final deverá ser produzida preferencialmente no formato **JPEG**, por ser adequado ao cenário de fotografias provenientes de webcam e por proporcionar boa relação entre qualidade visual e tamanho do arquivo.

O componente deverá produzir a fotografia como um objeto binário adequado ao ambiente web, preferencialmente um:

Blob

A aplicação poderá utilizar esse **Blob** diretamente para construir sua requisição HTTP, por exemplo por meio de `FormData`.

O componente não deverá exigir que a aplicação converta a fotografia para Base64 para utilizá-la.

17.3 Por que Blob

O **Blob** será a representação primária da fotografia produzida.

Essa escolha evita uma conversão desnecessária para Base64 e mantém a fotografia em uma representação binária apropriada para transporte HTTP.

Quando necessário, a aplicação poderá converter o **Blob** para outros formatos, como:

- `File`;
- Base64;
- `ArrayBuffer`;
- URL temporária para apresentação.

Entretanto, essas conversões não deverão ser responsabilidade obrigatória do `FotografoDeFACES`.

17.4 Fotografia confirmada

A fotografia capturada deverá ser disponibilizada pelo componente por meio de `value` e também por:

`getValue()`

Quando existir uma fotografia, ambos deverão representar a mesma fotografia válida do componente.

Quando não existir fotografia confirmada:

`value = null`

e:

`getValue() → null`

17.5 Recorte da face

A fotografia final deverá conter a face identificada pelo componente com uma margem de contexto.

O recorte não deverá ser excessivamente justo ao contorno facial.

Deverá existir espaço suficiente para preservar:

- toda a face;
- cabelos quando presentes;
- parte da cabeça;
- região próxima ao pescoço;
- pequeno contexto ao redor da face.

A margem deverá ser aplicada de forma consistente pelo componente.

A aplicação hospedeira não precisará conhecer ou recalcular a geometria do recorte.

17.6 Face não deverá ser cortada

O recorte deverá preservar integralmente a face selecionada.

Caso o enquadramento detectado não permita produzir uma fotografia adequada sem cortar partes relevantes da face, o componente não deverá considerar a captura PRONTO.

Essa regra é especialmente importante no modo quiosque, no qual o componente deverá ser flexível quanto ao posicionamento, mas não poderá sacrificar a integridade da fotografia.

17.7 Resolução de saída

A fotografia final deverá possuir uma resolução padronizada, independente da resolução original utilizada pela webcam.

Como referência inicial, o componente deverá trabalhar com uma fotografia final cujo lado maior esteja aproximadamente entre **640 e 800 pixels**, mantendo resolução suficiente para o processamento biométrico posterior sem produzir arquivos desnecessariamente grandes.

O valor exato deverá ser validado durante a implementação prática e os testes com o backend biométrico utilizado pelo projeto.

A resolução final deverá ser uma decisão interna do componente e não deverá depender de parâmetros fornecidos pela aplicação hospedeira em cada captura.

17.8 Qualidade JPEG

A fotografia deverá utilizar uma qualidade JPEG suficiente para preservar os detalhes faciais relevantes ao processamento posterior.

Como referência inicial, deverá ser utilizada uma qualidade próxima de:

0,85 a 0,92

O valor definitivo deverá ser estabelecido durante a implementação e validação.

A prioridade será encontrar o melhor equilíbrio entre:

- preservação dos detalhes faciais;
- tamanho do arquivo;
- velocidade de processamento;
- tempo de transferência;
- compatibilidade com equipamentos mais modestos.

17.9 Orientação da fotografia

A fotografia final deverá possuir orientação normalizada.

O componente deverá garantir que a imagem entregue à aplicação não dependa da orientação ou transformação utilizada internamente durante a captura.

A fotografia deverá ser entregue na orientação adequada para consumo pelo sistema.

17.10 Espelhamento

A visualização da câmera poderá ser espelhada para proporcionar uma experiência natural ao usuário, especialmente no modo autorretrato.

Entretanto, a fotografia final não deverá depender desse espelhamento visual.

A imagem produzida deverá possuir orientação consistente para utilização posterior pela aplicação e pelo backend.

Assim, a experiência de visualização e o conteúdo efetivamente produzido serão tratados como conceitos distintos.

17.11 Tratamento mínimo

O componente poderá realizar tratamentos necessários para padronizar a fotografia, tais como:

- recorte;
- redimensionamento;
- compressão JPEG;
- normalização de orientação;
- pequenas correções técnicas necessárias à preparação da imagem.

O componente não deverá realizar tratamentos agressivos que alterem artificialmente as características faciais.

Não fazem parte do objetivo:

- embelezamento;
- suavização estética da pele;
- alteração de traços;
- filtros artísticos;
- alteração de identidade visual.

A fotografia deverá representar fielmente a pessoa capturada.

17.12 Metadados

O `FotografoDeFACES` poderá manter informações técnicas relacionadas à fotografia produzida, mas a fotografia propriamente dita deverá permanecer simples e adequada ao consumo pela aplicação.

Informações como:

- resolução original;
- resolução final;
- dimensões do recorte;
- posição da face;

- qualidade avaliada;
- versão do algoritmo de preparação;

poderão ser disponibilizadas pela API de estado ou por informações técnicas relacionadas à captura.

Esses metadados não deverão ser obrigatoriamente incorporados ao arquivo JPEG.

17.13 Fotografia e backend

A aplicação hospedeira será responsável por decidir como utilizar a fotografia.

Por exemplo:

FotografoDeFaces

↓

Fotografia confirmada

↓

onChange / value

↓

Aplicação hospedeira

↓

FormData

↓

requisição HTTP

↓

Backend biométrico

O FotografoDeFaces não deverá conhecer nem controlar as etapas posteriores.

17.14 Fotografia para outros usos

A fotografia produzida não deverá ser considerada exclusiva para biometria.

Ela poderá ser utilizada pela aplicação em outros contextos, como:

- fotografia de perfil;
- cadastro de usuário;
- atualização cadastral;
- histórico de registros;
- auditoria;

- identificação visual;
- armazenamento documental;
- outros usos determinados pela aplicação.

Isso reforça a característica reutilizável do componente.

17.15 Contrato mínimo

O contrato mínimo da fotografia produzida será:

Formato: JPEG

Representação primária: Blob

Orientação: normalizada

Recorte: centralizado na face selecionada, com margem de contexto

Resolução: padronizada pelo componente

Compressão: adequada ao processamento facial

Conteúdo: uma única face selecionada

Tratamento: realizado pelo componente

Envio ao backend: responsabilidade da aplicação

17.16 Regra fundamental

O `FotografoDeFaces` deverá entregar à aplicação uma fotografia que possa ser considerada:

uma representação facial única, tratada, recortada, orientada, redimensionada e comprimida de maneira consistente, pronta para que a aplicação decida como utilizá-la.

A aplicação hospedeira não deverá precisar descobrir qual frame utilizar, recortar a face, determinar a margem, corrigir orientação ou preparar a imagem novamente antes de utilizá-la.

Esse é o limite final da responsabilidade do `FotografoDeFaces`.

18. Comportamentos de erro

O `FotografoDeFaces` deverá tratar internamente as situações previsíveis de falha relacionadas à câmera, detecção facial, avaliação da imagem, captura e preparação da fotografia.

O componente deverá diferenciar **condições normais de não prontidão de erros efetivos de execução**.

Uma face ainda não adequada, por exemplo, não constitui erro: significa apenas que o componente ainda não pode realizar a captura.

18.1 Princípio geral

O componente deverá evitar lançar exceções para situações esperadas durante seu funcionamento normal.

Situações como:

- nenhuma face detectada;
- mais de uma face no quiosque;
- face fora do enquadramento;
- iluminação insuficiente;
- movimento excessivo;
- face não frontal;
- candidato ainda não estável;
- captura solicitada antes de PRONTO;

deverão ser tratadas como condições operacionais do componente.

A aplicação poderá consultar a situação atual por meio de `getState()` e das demais consultas disponibilizadas pelo `useRef`.

18.2 Ausência de face

Quando nenhuma face válida for detectada, o componente deverá permanecer no ciclo de busca.

No modo autorretrato ou assistido, deverá orientar o usuário para posicionar o rosto adequadamente.

No modo quiosque, deverá permanecer aguardando uma pessoa elegível.

A ausência de face não deverá ser considerada uma falha de execução.

O estado deverá retornar ou permanecer em:

AGUARDANDO

ou avançar para DETECTANDO quando o componente estiver processando continuamente os frames da câmera.

18.3 Múltiplas faces

Nos modos autorretrato e assistido, a presença de mais de uma face deverá impedir que o componente alcance PRONTO.

O componente deverá orientar o usuário para que permaneça somente uma pessoa diante da câmera.

No modo quiosque, múltiplas faces poderão estar presentes.

Nesse caso, o componente deverá:

1. detectar as faces;
2. avaliá-las;
3. selecionar um único candidato conforme as regras definidas;
4. estabelecer Face Lock sobre esse candidato;
5. continuar o processo somente para ele.

A existência de outras pessoas na cena não deverá permitir que o componente alterne arbitrariamente o candidato enquanto estiver seguindo o fluxo para PRONTO.

18.4 Perda do candidato

Caso o candidato selecionado pelo Face Lock deixe de ser identificável, o componente deverá interromper o processo em andamento.

A situação deverá ser avaliada conforme a tolerância definida para estabilidade e continuidade da detecção.

Se a perda ultrapassar essa tolerância, o componente deverá abandonar o candidato e retornar a:

AGUARDANDO

Isso permitirá que uma nova pessoa seja selecionada.

No modo quiosque, essa regra é especialmente importante para impedir que o componente permaneça preso indefinidamente a uma pessoa que deixou o local.

18.5 Movimento durante o cronômetro

Se o componente estiver em CRONOMETRANDO e detectar movimento que comprometa os critérios de estabilidade, o cronômetro deverá ser interrompido.

O componente deverá abandonar a contagem e retornar ao processo de avaliação.

A partir daí, deverá novamente buscar as condições necessárias para iniciar outro ciclo de cronômetro.

A fotografia não deverá ser capturada enquanto a estabilidade necessária não estiver novamente garantida.

18.6 Falha na captura

Se ocorrer uma falha efetiva durante a captura, a fotografia não deverá ser considerada válida.

O componente deverá comunicar visualmente a situação ao usuário e disponibilizar uma mensagem apropriada.

A moldura de indicação poderá assumir a condição visual definida para falha, utilizando a cor vermelha quando a interface estiver configurada para apresentar esse recurso.

A aplicação poderá consultar o estado e a mensagem por meio do `useRef`.

Após o tratamento da falha, o componente deverá retornar ao ciclo de captura de maneira segura.

18.7 Falha no tratamento da fotografia

Se a fotografia for capturada, mas o processamento necessário para produzir o `Blob` final falhar, a fotografia não deverá ser confirmada.

O componente deverá:

6. descartar o resultado inválido;
7. informar a falha;
8. impedir que o resultado seja disponibilizado como fotografia confirmada;
9. retornar ao fluxo apropriado para uma nova tentativa.

Nenhum `onChange` deverá ser emitido para uma fotografia que não tenha sido efetivamente confirmada.

18.8 Falha de acesso à câmera

A câmera poderá não estar disponível por diferentes razões, incluindo:

- usuário não concedeu permissão;
- permissão foi revogada;
- câmera está sendo utilizada por outro processo;
- dispositivo não possui câmera;
- navegador não oferece acesso compatível;
- câmera deixou de responder;
- dispositivo foi desconectado.

Nessas situações, o componente deverá apresentar uma mensagem compreensível ao usuário.

Quando possível, a mensagem deverá orientar a ação necessária, como conceder permissão ou verificar a disponibilidade da câmera.

O componente não deverá entrar em um ciclo infinito de tentativas silenciosas.

18.9 Permissão negada

A negativa de permissão para utilização da câmera deverá ser tratada como condição de indisponibilidade da câmera.

O componente deverá informar claramente que o acesso à câmera não foi autorizado.

A aplicação hospedeira poderá consultar essa situação por meio de `getState()`.

O comportamento deverá permitir que a aplicação apresente orientações próprias, caso necessário.

18.10 Câmera indisponível após inicialização

A câmera poderá funcionar inicialmente e posteriormente deixar de estar disponível.

O componente deverá detectar a interrupção do fluxo de vídeo e atualizar seu estado.

A fotografia anteriormente confirmada, caso exista, não deverá ser apagada automaticamente apenas porque a câmera deixou de funcionar.

O componente deverá preservar o último estado válido da fotografia até que uma ação explícita determine sua substituição ou remoção.

18.11 `capture()` fora de PRONTO

Quando a aplicação chamar:

```
fotografoRef.current.capture();
```

e o componente não estiver em PRONTO, a solicitação deverá ser ignorada.

Não deverá ocorrer:

- captura forçada;
- fotografia inadequada;
- exceção como parte do fluxo normal;
- alteração indevida de `value`;
- `onChange`.

A aplicação poderá consultar `getState()` para determinar o motivo pelo qual a captura não estava disponível.

18.12 Erros internos inesperados

Falhas inesperadas de implementação não deverão ser silenciosamente mascaradas.

Quando apropriado, o componente deverá:

- registrar informações técnicas no mecanismo de diagnóstico disponível;
- preservar o último estado válido possível;
- evitar corromper a fotografia confirmada;
- impedir transições inconsistentes da máquina de estados;
- comunicar à aplicação uma condição de erro quando houver mecanismo apropriado para isso.

O mecanismo específico de diagnóstico será definido durante a implementação técnica.

18.13 Fotografia confirmada não deverá ser perdida por erro transitório

Uma regra fundamental será:

Um erro ocorrido durante uma nova tentativa de captura não deverá apagar ou corromper a última fotografia confirmada.

Por exemplo:

Fotografia A = confirmada

↓

Trocar

↓

nova captura

↓

falha

↓

Fotografia A continua sendo a fotografia válida

Somente uma ação explícita que resulte na confirmação de uma nova fotografia ou na confirmação da remoção poderá alterar o valor atualmente confirmado.

18.14 Erros e onChange

O onChange deverá ser acionado somente quando houver alteração efetiva da fotografia confirmada.

Portanto, não deverá ser disparado em razão de:

- falha na câmera;

- perda temporária da face;
- movimento;
- falha de captura;
- fotografia em avaliação;
- cancelamento;
- erro de processamento;
- tentativa de capture() antes de PRONTO.

A fotografia confirmada somente será alterada quando o fluxo de negócio definido pelo componente determinar essa alteração.

18.15 Erros e mensagens

As mensagens internas deverão permitir que o usuário compreenda o que está acontecendo.

Quando showMessage estiver habilitado, o componente poderá apresentar mensagens como:

Câmera indisponível.

Permita o acesso à câmera para continuar.

Nenhuma face detectada.

Mais de uma pessoa detectada. Aguarde.

Permaneça imóvel.

Não foi possível capturar a fotografia. Tente novamente.

As mensagens definitivas deverão ser consolidadas durante a implementação.

18.16 Erros não deverão alterar arbitrariamente a máquina de estados

Cada condição de erro deverá possuir uma transição conhecida.

O componente não deverá utilizar um estado genérico de erro para todas as situações sem distinguir sua origem.

De maneira geral:

Problema transitório

↓

retorna ao fluxo de avaliação

enquanto:

Problema que inviabiliza a captura

↓

estado apropriado de indisponibilidade

e:

falha durante captura/processamento

↓

resultado não confirmado

↓

novo ciclo

A máquina de estados deverá permanecer determinística.

18.17 Princípio de segurança do valor confirmado

O FotografoDeFACES deverá sempre priorizar a preservação da última fotografia confirmada.

Assim:

fotografia confirmada

↓

qualquer tentativa

↓

erro

↓

fotografia confirmada permanece válida

A substituição somente ocorrerá mediante o fluxo normal de confirmação.

18.18 Regra geral de recuperação

Sempre que possível, o componente deverá se recuperar automaticamente de condições transitórias.

Quando isso não for possível, deverá deixar a aplicação hospedeira em condições de compreender o problema por meio de:

- estado atual;
- mensagem;
- informações disponíveis no `useRef`.

O objetivo é evitar que a aplicação precise conhecer detalhes internos de câmera, detecção facial ou processamento de imagem para conseguir operar o componente corretamente.

19. Comportamento de reinício

O método `restart()` será o mecanismo oficial para solicitar ao `FotografoDeFACES` o encerramento do ciclo atual e o início de um novo ciclo de detecção.

O reinício deverá ser tratado como uma operação controlada pela máquina de estados e não como uma simples limpeza visual da interface.

19.1 Estado de destino

Independentemente do estado em que o componente se encontre, quando o reinício for aceito, seu novo ciclo deverá começar em:

AGUARDANDO

A partir desse estado, o componente deverá iniciar novamente o processo de busca e avaliação de uma face candidata.

Isso será particularmente importante no modo quiosque.

fotografia processada

↓

aplicação libera o componente

↓

`restart()`

↓

AGUARDANDO

↓

nova busca

↓

novo candidato

19.2 Reinício encerra o Face Lock

Ao executar `restart()`, qualquer Face Lock existente deverá ser obrigatoriamente desfeito.

O candidato anteriormente selecionado não terá mais prioridade.

No modo quiosque, isso significa que uma nova pessoa poderá ser escolhida.

Exemplo:

Pessoa A

↓

Face Lock A

↓

PRONTO

↓

captura

↓

`restart()`

↓

Face Lock A encerrado

↓

AGUARDANDO

↓

Pessoa B pode ser selecionada

Essa regra impede que o componente permaneça vinculado à pessoa anterior depois que seu ciclo tiver sido encerrado.

19.3 Reinício não significa apagar automaticamente a fotografia confirmada

O `restart()` deverá ser entendido como reinício do **processo de captura**, não necessariamente como remoção da fotografia atualmente confirmada.

Assim, a fotografia confirmada deverá permanecer preservada quando o reinício for utilizado para iniciar um novo ciclo de captura.

Por exemplo:

Fotografia A confirmada

↓

restart()

↓

AGUARDANDO

↓

Fotografia A continua sendo o valor confirmado

Isso permite que a aplicação reinicie o processo de captura sem perder o valor que já possui.

19.4 Diferença entre restart() e Limpar

restart() e a ação visual Limpar não deverão ser considerados necessariamente equivalentes.

restart() significa:

encerrar o ciclo atual de captura e iniciar um novo ciclo de detecção.

Limpar significa:

solicitar a remoção da fotografia atualmente confirmada dentro do fluxo de substituição definido pelo componente.

Portanto, uma ação Limpar poderá fazer parte de um fluxo de substituição e exigir Confirmar ou Cancelar.

Já restart() é um comando operacional destinado a reiniciar a captura.

19.5 Reinício durante AGUARDANDO

Se restart() for solicitado quando o componente já estiver em AGUARDANDO, o componente deverá permanecer nesse estado e garantir que seu ciclo de detecção esteja corretamente inicializado.

Não deverá haver comportamento adicional perceptível ao usuário.

19.6 Reinício durante DETECTANDO

O componente deverá:

1. interromper a avaliação atual;
2. descartar candidatos temporários;
3. desfazer qualquer seleção provisória;
4. retornar a AGUARDANDO;
5. iniciar um novo ciclo de detecção.

Nenhuma fotografia deverá ser produzida como consequência do ciclo interrompido.

19.7 Reinício durante AVALIANDO

Se houver uma face candidata em avaliação, ela deverá ser descartada como candidata atual.

O componente não deverá tentar preservar o candidato anterior.

Ao retornar para AGUARDANDO, uma nova seleção deverá ocorrer.

19.8 Reinício durante CRONOMETRANDO

O cronômetro deverá ser imediatamente cancelado.

O contador deverá ser zerado e não poderá concluir a captura posteriormente.

O fluxo deverá ser:

CRONOMETRANDO

↓

`restart()`

↓

cronômetro cancelado

↓

AGUARDANDO

Esse comportamento é essencial para impedir uma captura atrasada depois que a aplicação já tiver solicitado o reinício.

19.9 Reinício durante PRONTO

Se o componente estiver PRONTO, a condição de prontidão deverá ser abandonada.

A face atualmente selecionada deixará de ser o candidato bloqueado.

O componente retornará para:

AGUARDANDO

e iniciará uma nova busca.

Nenhuma captura automática pendente deverá ser executada após o reinício.

19.10 Reinício durante CAPTURANDO

Caso o reinício seja solicitado durante a captura, o componente deverá impedir que o resultado daquela operação seja considerado uma nova fotografia confirmada.

O resultado deverá ser descartado caso ainda não tenha sido finalizado e confirmado.

A operação deverá retornar ao ciclo de AGUARDANDO assim que for seguro fazê-lo.

O objetivo é impedir que uma captura iniciada antes do `restart()` altere o estado posteriormente de maneira inesperada.

19.11 Reinício durante FOTOGRAFIA_PRONTA

Se existir uma fotografia em avaliação, o reinício deverá abandonar essa fotografia em avaliação.

Ela não deverá substituir a fotografia confirmada.

A fotografia anteriormente confirmada, se existir, deverá continuar válida.

Fotografia A confirmada

Fotografia B em avaliação

↓

`restart()`

↓

Fotografia B descartada

↓

Fotografia A continua confirmada

↓

AGUARDANDO

19.12 Reinício durante erro ou indisponibilidade

Quando o componente estiver em uma condição recuperável de erro, `restart()` deverá ser uma das formas de solicitar uma nova tentativa.

O componente deverá limpar o contexto transitório da falha e iniciar novamente seu ciclo.

Quando a causa do problema persistir, o componente deverá permanecer na condição apropriada e informar a aplicação e/ou o usuário.

O `restart()` não deverá mascarar uma falha permanente.

19.13 Reinício e captura automática

Quando `autoCaptureAfter` estiver configurado, o reinício deverá cancelar qualquer captura automática que esteja pendente.

Por exemplo:

CRONOMETRANDO

"Aguarde 2 de 3 segundos"

↓

`restart()`

↓

cronômetro cancelado

↓

AGUARDANDO

O contador não poderá continuar em segundo plano.

Somente um novo ciclo, iniciado após AGUARDANDO, poderá criar um novo cronômetro.

19.14 Reinício e fotografia confirmada

A fotografia confirmada deverá ser tratada como um dado estável do componente.

Por padrão:

`restart()`

↓

não altera value

Assim, o reinício do processo de captura não deverá provocar `onChange`.

A fotografia somente será alterada quando ocorrer uma operação que efetivamente modifique a fotografia confirmada.

19.15 Reinício e `onChange`

A simples chamada:

```
fotografoRef.current.restart();
```

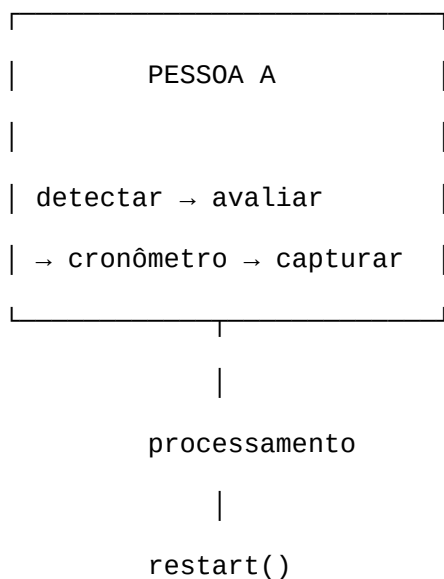
não deverá disparar `onChange`.

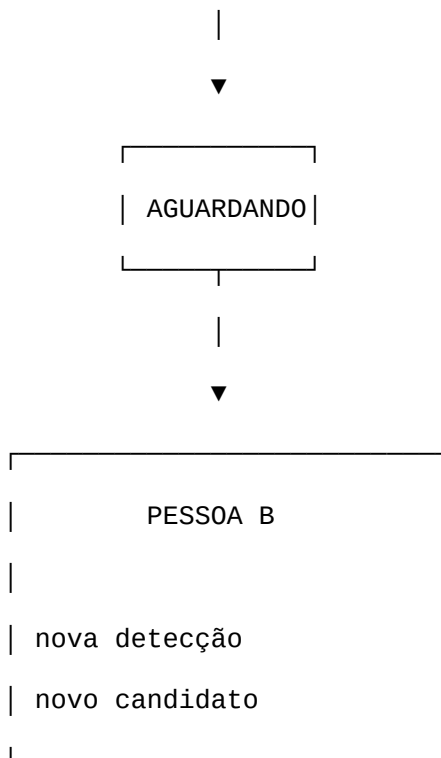
O evento somente será disparado se, como consequência de uma operação explicitamente confirmada, o valor da fotografia atualmente confirmada realmente mudar.

19.16 Reinício no modo quiosque

No modo quiosque, `restart()` será particularmente importante para estabelecer a fronteira entre duas pessoas.

O componente deverá tratar o reinício como o encerramento definitivo do ciclo do candidato anterior.





Somente depois do retorno a AGUARDANDO uma nova pessoa poderá assumir o foco do componente.

19.17 Reinício não deverá depender da pessoa anterior

Depois de `restart()`, o componente não deverá tentar localizar novamente a pessoa que estava anteriormente selecionada.

Mesmo que ela permaneça diante da câmera, deverá ser tratada como uma nova candidata em um novo ciclo.

Isso torna o comportamento determinístico e simplifica a integração com aplicações de ponto, cadastro e outros fluxos sequenciais.

19.18 Regra fundamental

O comportamento de `restart()` poderá ser resumido assim:

`restart()` encerra o ciclo atual, cancela operações transitórias, desfaz o Face Lock, interrompe cronômetros pendentes, abandona fotografias em avaliação e devolve o componente ao estado AGUARDANDO, preservando a última fotografia confirmada.

Essa definição deverá ser utilizada como referência durante a implementação da máquina de estados.

20. Limites entre componente e aplicação

O `FotografoDeFaces` deverá possuir responsabilidades claramente delimitadas em relação à aplicação hospedeira.

O componente será responsável pelo processo de captura e preparação da fotografia, enquanto a aplicação será responsável por decidir o que fazer com o valor produzido.

20.1 Responsabilidades do `FotografoDeFaces`

O componente será responsável por:

- acessar e utilizar a câmera conforme as configurações e permissões disponíveis;
- detectar faces;
- selecionar a candidata conforme as regras definidas;
- controlar o `Face Lock`;
- avaliar qualidade;
- controlar estabilidade;
- determinar o estado `PRONTO`;
- controlar o cronômetro;
- realizar a captura;
- tratar e preparar a fotografia;
- manter a máquina de estados;
- controlar a apresentação visual;
- disponibilizar a fotografia por meio de `value`;
- comunicar alterações de `value` por meio de `onChange`;
- controlar temporariamente `value_rollback` quando houver uma operação de alteração;
- disponibilizar os comandos e consultas definidos pela API `useRef`.

20.2 Responsabilidades da aplicação hospedeira

A aplicação será responsável por:

- fornecer o `value` inicial ou atual;
- manter o estado externo da fotografia;
- tratar o evento `onChange`;
- decidir se e quando a fotografia deverá ser enviada ao backend;
- realizar reconhecimento facial ou outros processamentos biométricos;
- decidir o que fazer com o resultado do processamento;
- decidir quando o componente deverá ser reiniciado;
- controlar propriedades reativas por meio do fluxo normal do React;
- definir as regras de negócio relacionadas à utilização da fotografia.

O componente não deverá conhecer as regras de negócio da aplicação.

20.3 Modelo controlado

O `FotografoDeFaces` deverá seguir o padrão de componente controlado do React.

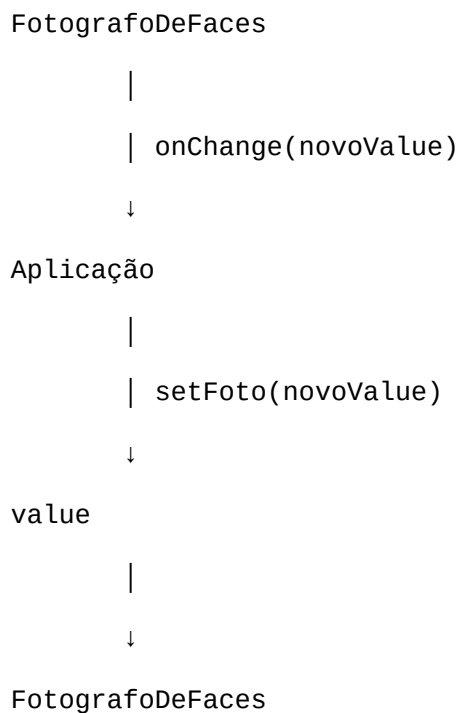
A aplicação deverá manter o valor da fotografia:

```
const [foto, setFoto] = useState(null);
```

e fornecê-lo ao componente:

```
<FotografoDeFaces  
  value={foto}  
  onChange={setFoto}  
>
```

O fluxo será:



O `value` recebido pelo componente deverá representar sempre o valor atual da fotografia.

O componente não deverá criar um significado próprio para esse valor, como "fotografia confirmada".

Quando precisar alterar o valor, deverá comunicar a alteração por `onChange`, cabendo à aplicação atualizar seu estado e fornecer novamente o novo `value`.

20.4 Exemplo de configuração

Um exemplo conceitual de utilização poderá ser:

```
const [foto, setFoto] = useState(null);
```

```
<FotografoDeFaces
  value={foto}
  onChange={setFoto}
  mode="autorretrato"
  autoCaptureAfter={3000}
  reviewFor={3000}
  showMessages={true}
  showFaceFrame={true}
  showFramingGuide={true}
  cameraId={null}
/>
```

Nesse exemplo:

- `value` inicia como `null`;
- `onChange` mantém a aplicação sincronizada com o valor atual do componente;
- `mode` define o modo de operação;
- `autoCaptureAfter` define o comportamento de captura automática;
- `reviewFor` define a janela de tempo durante a qual as ações de revisão poderão ser apresentadas;
- `showMessages` controla a exibição das mensagens internas;
- `showFaceFrame` controla a exibição da moldura da face;
- `showFramingGuide` controla a exibição da guia fixa de enquadramento;
- `cameraId` permite indicar a câmera a ser utilizada quando aplicável.

20.5 Fotografia inicial

A aplicação poderá iniciar o componente já contendo uma fotografia:

```
const [foto, setFoto] = useState(fotoExistente);
```

```
<FotografoDeFaces  
    value={foto}  
    onChange={setFoto}  
    mode="autorretrato"  
    reviewFor={3000}  
/>
```

Nesse caso, o componente deverá apresentar a fotografia recebida em `value` e ajustar seu comportamento ao valor atual recebido.

Não deverá ser necessário realizar uma nova captura para que a fotografia existente seja apresentada.

20.6 Alteração da fotografia

Quando o componente produzir uma nova fotografia ou quando uma ação do usuário resultar em alteração do valor atual, o componente deverá utilizar `onChange`.

Por exemplo:

```
Nova fotografia  
    ↓  
onChange(novaFoto)  
    ↓  
setFoto(novaFoto)  
    ↓  
value = novaFoto
```

A aplicação poderá então enviar a fotografia ao backend, armazená-la, submetê-la a processamento biométrico ou realizar qualquer outra operação de negócio.

Essas decisões não fazem parte das responsabilidades do `FotografoDeFaces`.

20.7 Operações de Trocar, Limpar, Confirmar e Cancelar

Quando `reviewFor` estiver configurado, o componente poderá oferecer ao usuário as operações de revisão previstas neste documento.

O componente será responsável pela dinâmica interna dessas operações, incluindo o uso temporário de `value_rollback`.

A aplicação continuará responsável pelo `value` público.

Assim, por exemplo, uma alteração poderá resultar em:

```
value = Foto A
```

```
value_rollback = null
```

↓ Trocar

```
value = null
```

```
value_rollback = Foto A
```

↓ nova captura

```
value = Foto B
```

```
value_rollback = Foto A
```

↓ Confirmar

```
value = Foto B
```

```
value_rollback = null
```

Ou, em caso de cancelamento:

```
value = Foto B
```

```
value_rollback = Foto A
```

↓ Cancelar

onChange(Foto A)

↓

value = Foto A

value_rollback = null

O componente não deverá enviar `value_rollback` para a aplicação como parte de sua API pública.

20.8 Limite da responsabilidade biométrica

O `FotografoDeFaces` não deverá realizar:

- reconhecimento da identidade da pessoa;
- comparação biométrica;
- geração de embeddings;
- consulta a banco de dados biométrico;
- autenticação ou autorização;
- envio da fotografia ao serviço biométrico;
- interpretação do resultado de um serviço biométrico.

Sua responsabilidade termina na produção e disponibilização da fotografia conforme o contrato definido.

A aplicação hospedeira será responsável por utilizar essa fotografia conforme sua finalidade.

20.9 Limite da responsabilidade sobre regras de negócio

O componente não deverá decidir:

- quem pode ser fotografado;
- por que a fotografia está sendo capturada;
- se uma fotografia poderá ser armazenada;
- se uma fotografia deverá substituir outra no banco de dados;
- se uma fotografia deverá ser submetida a reconhecimento facial;
- se uma pessoa deverá ser autorizada ou recusada;
- o que fazer quando o reconhecimento facial produzir determinado resultado.

Essas decisões pertencem exclusivamente à aplicação hospedeira.

20.10 Dimensionamento e apresentação

O `FotografoDeFaces` deverá adaptar sua apresentação à área disponibilizada pelo elemento hospedeiro, sem exigir que a aplicação forneça propriedades específicas para largura ou altura.

O dimensionamento do componente deverá ser determinado pelo contexto em que ele for utilizado.

A aplicação hospedeira será responsável por definir o espaço destinado ao componente por meio do elemento contêiner e de seus mecanismos normais de layout, como dimensões, `flex`, `grid` ou outras regras de CSS.

O `FotografoDeFaces` deverá ocupar adequadamente a área que lhe for disponibilizada, preservando a proporção da imagem da câmera e evitando deformações.

Comportamento visual

A área de vídeo deverá adaptar-se ao espaço disponível no componente.

O conteúdo da câmera deverá:

- ocupar adequadamente a área disponível;
- preservar a proporção original da imagem;
- evitar distorção da face ou do vídeo;
- manter a guia de enquadramento centralizada quando `showFramingGuide` estiver habilitada;
- manter a moldura da face posicionada de acordo com a face detectada;
- permitir que os elementos visuais sobrepostos acompanhem corretamente a área efetiva de vídeo.

A adaptação do componente não deverá alterar a lógica de detecção, enquadramento ou avaliação da face.

Responsabilidade do hospedeiro

O dimensionamento externo deverá ser definido pela aplicação.

Exemplo conceitual:

```
<div className="area-da-camera">
    <FotografoDeFaces ... />
</div>
```

Nesse caso, o `FotografoDeFaces` deverá adaptar-se à área disponibilizada por `area-da-camera`.

A aplicação poderá utilizar diferentes dimensões conforme o contexto:

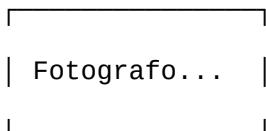
Quiosque



Tablet



Área reduzida



O componente deverá funcionar adequadamente nos diferentes tamanhos sem que seja necessário alterar suas propriedades de largura ou altura.

Ausência de propriedades específicas de dimensão

Não deverão ser criadas propriedades específicas como:

`width`

`height`

`cameraWidth`

`cameraHeight`

O dimensionamento deverá permanecer uma responsabilidade do elemento hospedeiro e de seu sistema de layout.

Essa decisão mantém o componente mais reutilizável e evita que sua API seja contaminada por propriedades de apresentação que podem ser resolvidas adequadamente pelo CSS e pelo contexto de utilização.

Tela cheia

Quando a aplicação solicitar:

```
fotografoRef.current.setFullscreen(true)
```

o componente deverá adaptar sua apresentação ao espaço de tela cheia disponível, mantendo as mesmas regras de proporção, enquadramento e sobreposição visual.

Ao sair da tela cheia, deverá novamente adaptar-se à área fornecida pelo hospedeiro.

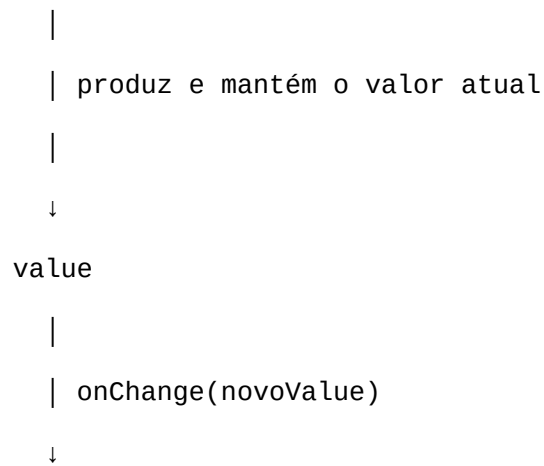
Critérios de aceite

1. O `FotografoDeFaces` deverá adaptar-se à área fornecida pelo elemento hospedeiro.
2. O componente não deverá exigir propriedades específicas de largura ou altura.
3. A aplicação deverá poder dimensionar o componente utilizando os mecanismos normais de CSS e layout.
4. O vídeo deverá preservar sua proporção e não poderá apresentar deformação causada pelo dimensionamento.
5. A guia de enquadramento, quando habilitada, deverá permanecer centralizada na área efetiva do componente.
6. A moldura da face deverá permanecer corretamente posicionada sobre a face detectada após o redimensionamento do componente.
7. O redimensionamento da área hospedeira não deverá alterar os critérios utilizados para determinar se uma face está PRONTO.
8. O comportamento deverá permanecer funcional em diferentes dimensões de tela e dispositivos.
9. O modo de tela cheia deverá utilizar a área disponível sem exigir propriedades adicionais de largura ou altura.
10. O componente deverá manter separadas as responsabilidades de **dimensionamento da interface**, pertencentes ao hospedeiro, e de **captura, enquadramento e processamento da face**, pertencentes ao `FotografoDeFaces`.

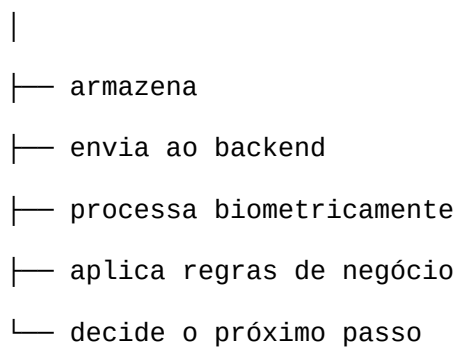
20.11 Princípio geral

O limite entre o `FotografoDeFaces` e a aplicação hospedeira poderá ser resumido da seguinte forma:

FotografoDeFaces



Aplicação hospedeira



O `FotografoDeFaces` deverá cuidar de **capturar e preparar a fotografia**.

A aplicação hospedeira deverá cuidar de **dar finalidade à fotografia**.

Esse limite deverá ser preservado para que o componente permaneça reutilizável em diferentes contextos, como captura inicial, atualização de perfil, substituição de fotografia e histórico de registros.